

Manuscript prepared for IEEE Access submission.

Federated Double Deep Q-Network for Channel-Aware Task Offloading with Rejection-Aware Adaptive Resource Allocation in Edge-Cloud IoT Systems

MAYANK LOHANI¹, SARANYA G², and KUMARAN K³¹

¹Affiliation details to be supplied after author verification

²Affiliation details to be supplied after author verification

³Affiliation details to be supplied after author verification

Corresponding author: Mayank Lohani.

DOI:

Abstract—Task offloading in edge-cloud Internet of Things (IoT) systems changes with task urgency, channel quality, edge queue pressure, cloud load, and fault state. Policies trained on mostly independent task samples can miss the behavior that appears during bursty arrivals, outage windows, queue carryover, and infeasible tasks. This article proposes a two-stage scheduling method for channel-aware offloading and resource management. Stage 1 uses a zone-aware Federated Double Deep Q-Network (Fed-DDQN) to choose between edge and cloud execution across non-identical edge zones. Rejection is modeled as an admission or infeasibility outcome, not as a third action. Stage 2 assigns CPU, memory, and bandwidth to edge-selected tasks through a rejection-aware residual allocator. The method is evaluated on the benchmark dataset, a realistic synthetic edge-cloud IoT benchmark with 100,000 tasks, temporal workload bursts, congestion, outage, jitter, edge degradation, and cloud maintenance. On the untouched temporal test split, Fed-DDQN achieved 136.009 ms average latency, 97.73% SLA satisfaction, 3.68% rejection, and 67.43% edge usage. Its latency was 6.15% lower than centralized DDQN and 1.71% lower than an FL-DDPG-style baseline; the latency-reference Oracle reached 123.310 ms. On the allocator-valid edge-selected subset, the hand-designed rejection-aware demand rule gave the lowest proxy-target MAE, while residual learning with risk projection and capacity normalization gave the best efficiency score, lowest under-allocation, and zero capacity violation. The results indicate that the proposed pipeline improves latency and QoS balance under the evaluated edge-cloud workload conditions, while the latency-reference Oracle gap leaves clear room for further optimization.

Index Terms—Double Deep Q-Network, edge-cloud computing, federated reinforcement learning, Internet of Things, quality of service, resource allocation, task offloading.

1 INTRODUCTION

Edge-cloud Internet of Things (IoT) systems give a scheduler two imperfect execution sites. Edge nodes

sit close to devices and can avoid long communication paths, but their CPU, memory, queue, and energy budgets are limited. Cloud nodes provide larger compute pools, yet cloud execution depends on network delay, uplink capacity, packet loss, jitter, cold starts, and maintenance state. The scheduling decision is not a fixed preference for edge or cloud; it is repeated under changing resource and channel conditions.

The problem becomes harder when arrivals are bursty. A camera-heavy interval, a firmware rollout, or an emergency burst can make many devices compete for the same edge resources at nearly the same time. A latency-only policy can look strong on average and still fail in these windows. Routing too much work to edge may reduce communication delay while increasing queue delay and admission-failure risk. Routing too much work to cloud may avoid local overload but miss tight deadlines when the network is degraded. A usable policy has to learn these tradeoffs without assuming that every zone sees the same workload distribution.

Deep reinforcement learning (DRL) is a natural fit for this setting because task offloading is sequential and state dependent. Double Deep Q-Networks (DDQNs) are especially relevant when the action space is discrete, because the double target reduces overestimation compared with a single Q-network. A centralized DDQN, however, assumes that all training data can be pooled and that one global distribution is enough. Edge-cloud IoT systems often violate both assumptions. Different edge zones can see different task types, channel profiles, device classes, and failure patterns. Federated learning offers a better training structure: each zone can train locally on its own distribution, and the server aggregates model updates rather than raw data.

This work studies task offloading as a federated DDQN problem. The Stage-1 model is a zone-aware Fed-DDQN policy for binary edge-cloud offloading. The action space is deliberately small: $a=0$ selects edge execution and $a=1$ selects cloud execution. Rejection is not a third action. It is a derived admission or infeasibility outcome that occurs when the selected path cannot satisfy resource, deadline, security, or state constraints. This distinction matters. If rejection were an action, a scheduler could appear cleaner by refusing

difficult tasks. In the binary setting, the policy must decide where feasible work should go.

The paper also includes a second-stage resource allocation model. After Fed-DDQN chooses edge execution, a rejection-aware residual allocator assigns CPU, memory, and bandwidth shares. The allocator starts from a demand-proportional allocation, learns deadline, priority, and QoS/admission-risk corrections, and then applies capacity normalization. It does not replace the offloading policy. It manages resources for tasks that the proposed scheduler has already selected for local execution.

The evaluation uses the benchmark dataset, a realistic synthetic benchmark built for controlled comparison. The benchmark is not measured deployment telemetry. It is more stressful than independent random task rows because it includes workload phases, temporal bursts, outages, jitter storms, congestion, edge degradation, cloud maintenance, and queue carryover. The fixed dataset keeps comparisons repeatable while still exposing policies to deployment-like variation.

On the untouched temporal test split, Fed-DDQN achieved 136.009 ms average latency, 97.73% SLA satisfaction, 3.68% rejection, and 67.43% edge usage. Its latency is 6.15% lower than centralized DDQN and 1.71% lower than the FL-DDPG-style baseline. The proposed model gives the lowest latency among the main comparison methods reported in Table 12 while preserving a federated, zone-aware training design.

Three questions guide the study. First, can federated DDQN reduce test latency compared with centralized DDQN and the implemented offloading baselines when the workload is evaluated in a future temporal interval? Second, can a reward that penalizes SLA miss and admission failure avoid the failure mode in which mean latency improves only because difficult tasks become infeasible? Third, once the destination is selected, can a residual allocator improve CPU, memory, and bandwidth assignment without changing that destination? These questions determine the paper structure: the method separates offloading from allocation, and the results report latency, QoS, rejection, scenario behavior, ablation, and allocation metrics rather than a single latency number.

The contributions are as follows:

- A zone-aware Fed-DDQN framework for binary edge-cloud IoT task offloading under non-identical edge-zone distributions.
- A channel-aware and rejection-aware reward design that accounts for latency, SLA misses, infeasibility risk, edge overuse, and heavy-task misrouting.
- A fixed-schema realistic synthetic benchmark, the benchmark dataset, with temporal bursts, contention effects, network faults, edge degradation, and cloud maintenance.
- A two-stage decision pipeline in which Fed-DDQN selects edge or cloud execution and a residual allocator refines CPU, memory, and bandwidth shares for edge-selected tasks.
- A test-only evaluation against a faithful DDQN baseline, a federated actor-critic-style comparator, several style-inspired scheduling and metaheuristic comparators, and a non-trainable latency-reference Oracle, with scenario-wise, multi-seed, ablation, and allocator analyses.

The manuscript proceeds as follows. Section 2 reviews edge-cloud offloading, DRL, federated learning, and resource allocation. Section 3 defines the system model and optimization objective. Section 4 describes the benchmark dataset and the feature pipeline. Section 5 presents the proposed federated DDQN offloading method. Section 6 describes the resource allocator. Section 7 gives the experimental protocol, and Section 8 reports the results and discussion.

2 RELATED WORK

2.1 Edge-Cloud IoT Task Offloading

Mobile edge computing places compute and storage resources closer to users, devices, and sensors. Moving selected tasks away from distant cloud centers can reduce response time and improve local context awareness [1, 2]. The same architecture creates a scheduling problem. Edge resources are finite, and the network between devices, edge nodes, and cloud nodes is not stable. A task that is best served by the edge during light load may be better served by the cloud when the local queue grows.

Classical offloading studies model the communication-computation tradeoff directly. Binary offloading, energy-aware offloading, and joint radio-compute allocation have been studied with optimization methods and queue-aware formulations [3–5]. These methods provide a strong analytical foundation. They also make the cost structure explicit: local execution depends on device or edge cycles, remote execution depends on transmission rate and server availability, and the final decision should respect latency and energy constraints. Their limitation in the present setting is not that the models are wrong, but that their assumptions can become brittle under bursty IoT traces, edge faults, cloud maintenance, and non-identical regional workloads.

Recent surveys and task-offloading studies show a clear movement from static optimization toward learning-based online scheduling [2, 6]. The reason is practical. In a real deployment, the scheduler may not know the exact future queue, channel, and task mix. A learning policy can adapt from observations. The open question is how to train such a policy when the data are naturally distributed across edge zones and when difficult tasks may be rejected because of deadline, battery, security, or resource constraints.

2.2 Deep Reinforcement Learning for Offloading

DRL is widely used for online offloading because it can map high-dimensional states to actions without requiring a closed-form solution for every operating condition [6–8]. In these methods, the state often contains channel quality, queue length, task size, available compute, and energy indicators. The reward usually penalizes latency, energy consumption, or a weighted combination of both. This work keeps the

broad MDP framing but gives explicit attention to rejection and SLA behavior, since a scheduler that reduces mean latency while increasing infeasible cases is not acceptable for QoS-sensitive IoT services.

DDQN fits binary or small discrete action spaces because it separates action selection from target evaluation, reducing the overestimation problem that can appear in standard DQN [9, 10]. Dueling Q-networks further separate the state value and the action advantage, which helps when a state is favorable or poor regardless of the chosen action [11]. These properties match the Stage-1 offloading decision, where the action is binary and many states are dominated by task urgency or network quality.

Actor-critic methods and deterministic policy gradients are often used when the action is continuous or when resource allocation is optimized together with offloading [12]. For that reason, the experimental comparison includes an FL-DDPG-style baseline. The difference in our design is deliberate: the proposed Fed-DDQN focuses on the binary offloading decision, and continuous-like CPU, memory, and bandwidth assignment is handled by a separate Stage-2 allocator. This split keeps the reinforcement-learning action space small and makes it easier to interpret the offloading decision itself.

2.3 Federated Learning and Non-IID Edge Zones

Federated learning trains a shared model from distributed clients without directly pooling raw local data [13, 14]. FedAvg remains the basic aggregation rule, while FedProx adds a proximal term to reduce client drift under heterogeneous data distributions [13, 15]. Edge-cloud IoT is a natural setting for such methods because local zones can differ in task types, radio quality, device classes, and failure patterns.

Federated reinforcement learning has recently been applied to task offloading and resource allocation in mobile edge and wireless IoT systems [16–19]. These studies motivate the present work, but the evaluation focus here is different. Instead of treating federated learning mainly as a training architecture, the study asks whether a federated DDQN policy can make reliable edge-cloud decisions under deployment-inspired stress. The benchmark includes non-IID zones, workload phases, network outages, congestion windows, edge degradation, and cloud maintenance, so the federated setting affects the decision problem as well as the training layout.

2.4 Joint Offloading and Resource Allocation

Resource allocation is closely tied to offloading. A cloud or edge decision is incomplete if the scheduler ignores CPU cycles, memory demand, bandwidth, and queue pressure [3–5]. Several studies optimize offloading and resource allocation jointly, often with energy, delay, or throughput objectives. Joint optimization can be powerful, but it can also produce a large action space and make it harder to isolate whether the offloading policy or the resource allocation rule caused a result.

The proposed method uses a two-stage formulation. The first stage learns the edge/cloud choice with a

binary Fed-DDQN policy. The second stage applies a residual allocator only to edge-selected tasks. The allocator cannot hide a poor offloading policy by changing the destination. It can only improve proxy-target allocation fit for tasks already routed to the edge. That separation keeps the proposed offloading model clear while still adding a practical resource-management step.

2.5 Position of This Work

The closest prior line of work combines DRL, MEC offloading, and federated training [16–19]. The present work differs in four ways. First, it uses a fixed synthetic benchmark with burst, outage, queue, and maintenance effects instead of independent task samples. Second, it evaluates rejection as an outcome, not as an optional action that can be used to avoid difficult tasks. Third, it uses purged temporal train, validation, and test splits so the final comparison is not tuned on the test interval. Fourth, it adds a post-offloading allocator and evaluates that allocator separately from the Stage-1 policy. These choices give a controlled basis for comparing offloading behavior, QoS outcomes, and resource-allocation quality.

3 SYSTEM MODEL AND PROBLEM FORMULATION

3.1 Edge-Cloud IoT Architecture

The system contains IoT devices, edge nodes, cloud nodes, and a scheduling broker. Devices generate tasks over discrete timesteps. Each task is associated with one nearby edge node, but the scheduler can choose either edge execution or cloud execution. Edge execution uses local compute and memory near the device. Cloud execution uses remote capacity and includes uplink, network, queue, and cloud processing delay.

Let $\mathcal{E} = \{1, \dots, E\}$ denote the set of edge nodes and $\mathcal{C} = \{1, \dots, C\}$ denote the set of cloud nodes. In the benchmark dataset, $E = 50$ and $C = 10$. Let $t \in \{1, \dots, T\}$ be the timestep, with $T = 1000$. Edge node j at time t has CPU availability $u_{j,t}^e$, memory availability $m_{j,t}^e$, queue length $q_{j,t}^e$, energy state $\eta_{j,t}^e$, and fault state $f_{j,t}^e$. The network state at time t is n_t , which includes uplink bandwidth, downlink bandwidth, packet loss, signal-to-noise ratio (SNR), base delay, outage state, congestion state, and jitter state. Cloud node k has CPU availability $u_{k,t}^c$, memory availability $m_{k,t}^c$, queue length $q_{k,t}^c$, and maintenance or overload indicators.

3.2 Task Model

A task i arriving at time t_i is represented by $\tau_i = (d_i, c_i, m_i, \delta_i, p_i, s_i, b_i, \kappa_i, t_i, j_i)$, where d_i is input size, c_i is CPU demand, m_i is memory demand, δ_i is the deadline, p_i is priority, s_i captures security sensitivity, b_i captures battery state, κ_i is task type, and j_i is the assigned edge node. Task type matters because emergency, video, telemetry, and firmware tasks can have different urgency and compute patterns even at similar sizes.

The task and state columns are generator-native quantities rather than direct measurements from one physical deployment. Size, demand, bandwidth, and availability are converted into normalized pressure or

Table 1: Positioning of the proposed work relative to related edge-cloud offloading research.

Category	Representative focus	Common limitation for this study	How this paper differs
Optimization-based MEC offloading	Binary offloading, radio-compute allocation, energy and latency tradeoffs [3–5]	Often relies on tractable model assumptions and may not adapt to burst/fault state changes.	Uses learned offloading under generated temporal bursts, congestion, queue carryover, and fault windows.
DRL offloading	DQN, DDQN, actor-critic, and online scheduling policies [6–8]	Many evaluations emphasize average latency or energy without explicit rejection-aware analysis.	Uses reward and reporting that include SLA miss, rejection, edge pressure, and heavy-task routing behavior.
Federated edge intelligence	FedAvg, FedProx, and federated DRL across distributed clients [13, 15–19]	Federated structure can be shown without strong stress-test evidence or scenario-level diagnostics.	Trains across non-IID edge zones and reports scenario-wise, multi-seed, and ablation evidence.
Joint offloading and allocation	Coupled destination selection and resource assignment [3–5]	Large joint action spaces can obscure whether gains come from routing or allocation.	Separates Stage-1 binary offloading from Stage-2 residual resource allocation for interpretability.
Benchmark and simulation studies	Synthetic tasks, simulated MEC nodes, and replayed state tables [1, 2]	Independent samples can under-state contention and race-like load effects.	Uses the benchmark dataset with fixed schemas, time-correlated stress windows, and precomputed contention pressure.

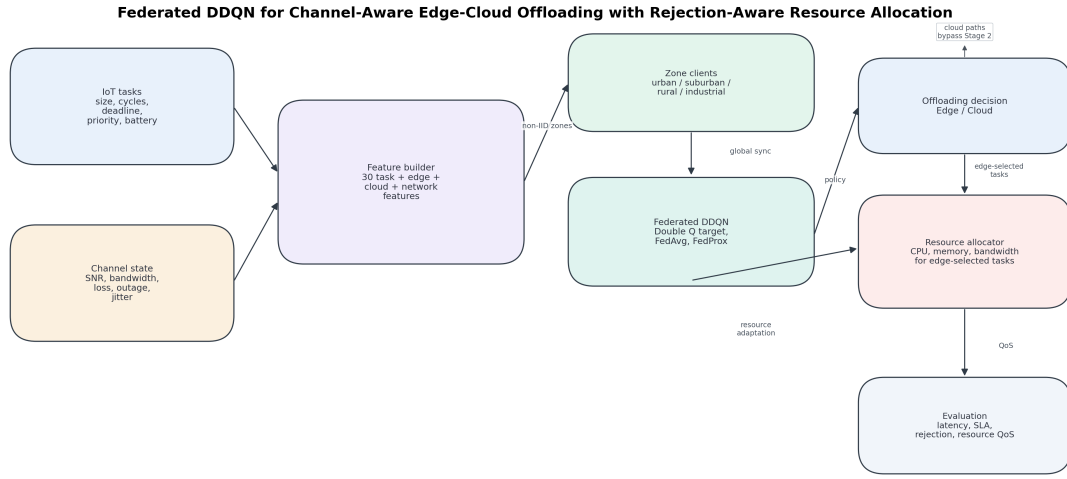


Figure 1: Two-stage edge-cloud IoT system. Stage 1 is the proposed federated DDQN offloading policy. Stage 2 applies rejection-aware resource allocation only to tasks selected for edge execution.

Table 2: Main notation for the edge-cloud offloading problem.

Symbol	Meaning
τ_i	Task i with size, compute, memory, deadline, priority, security, battery, type, arrival time, and assigned edge node.
κ_i	Task type/category, such as emergency, video, AI, telemetry, or firmware update.
t_i	Arrival timestep of task i .
j_i	Edge node associated with task i by locality.
a_i	Binary offloading action, where 0 denotes edge execution and 1 denotes cloud execution.
F_i	Engineered feature vector used by the Stage-1 policy.
L_i^e, L_i^c	Estimated edge and cloud latency for task i .
$\tilde{L}_i(a_i), \tilde{L}_i(a_i)$	Selected latency and normalized selected latency under action a_i .
$L_i^{\text{best}}, L_i^{\text{worst}}$	Minimum and maximum finite generated latency over edge/cloud alternatives after infeasible sentinel values are excluded for reward shaping.
δ_i	Task-specific deadline or SLA threshold.
M_i	SLA-miss indicator, equal to $1 - \text{SLA}_i$.
$\Phi_i^e, \Phi_i^c, \Phi_i(a_i)$	Edge, cloud, and selected-path feasibility predicates used to compute selected-path rejection.
F_i	Selected-path feasibility or admission failure indicator.
R_i	Reported rejection indicator, equal to selected-path feasibility or admission failure F_i .
Q_i	Broad QoS-failure indicator, $Q_i = \mathbb{I}[M_i = 1 \vee F_i = 1]$, used only when a union of SLA miss and rejection is needed.
$\hat{\rho}_i$	Pre-decision QoS/admission-risk estimate from observable task, channel, queue, and feasibility predictors; distinct from realized outcomes M_i, R_i , and Q_i .
$u_i^{\min}, \bar{m}_i$	Normalized CPU and memory feasibility thresholds for task i in the generated resource-availability scale.
$u_{j,t}^e, m_{j,t}^e$	Edge CPU and memory availability for node j at time t .
$q_{j,t}^e$	Edge queue length for node j at time t .
n_t	Network state at time t , including bandwidth, delay, packet loss, SNR, outage, congestion, and jitter indicators.
θ, θ^-	Online and target Q-network parameters.
θ_z, θ_g	Local zone model and current global federated model.
$\mathbf{y}_i^*$	Generated Stage-2 proxy target for CPU, memory, and bandwidth allocation on allocator-valid edge-selected tasks.
$\tilde{\mathbf{y}}_i$	Stage-2 predicted/final CPU, memory, and bandwidth allocation for edge-selected task i .
$\mathcal{G}_e^{\text{valid}}(j, t)$	Allocator-valid edge-selected tasks sharing edge node j and timestep t for Stage-2 group-wise capacity normalization.

Table 3: Additional method notation and implementation parameters.

Symbol	Meaning in this benchmark implementation
$u_i^{\min}$	Minimum normalized edge CPU availability required by task i after converting demand to the generator-native availability scale.
$\bar{m}_i$	Minimum normalized edge memory availability required by task i after converting memory demand to the generator-native availability scale.
P_i^e	Edge-pressure penalty used when local queue, edge overload, or edge-overuse risk makes an edge decision less desirable.
H_i	Heavy-task edge-misrouting indicator for AI, video, image, and firmware-like tasks when cloud execution is predicted to be safer or faster.
h_i	Compact heavy-task flag used in margin and allocator formulas; $h_i = 1$ for cloud-favorable heavy task categories and 0 otherwise.
$\bar{\delta}_i$	Deadline-tightness score for task i ; larger values mean tighter deadlines after normalization.
χ_i^{rt}	Binary real-time task flag used in Stage-2 urgency and allocation-target generation.
$\mathcal{G}_i^{\text{valid}}(j, t)$	Allocator-valid edge-selected task group sharing edge node j and timestep t in the Stage-2 normalization pass.
$C_{j,t}^r$	Normalized available capacity of edge node j for resource channel $r \in \{\text{cpu}, \text{mem}, \text{bw}\}$ at time t .
ϵ	Small positive constant used in denominators and projection formulas to avoid division by zero.
$\alpha_{\text{res}}, \alpha_{\text{dem}}$	Fixed blend weights for the Stage-2 residual and rejection-aware demand branches; the implementation uses 0.10 and 0.90, respectively.
ω_i	Bounded Stage-2 demand multiplier derived from urgency and pre-decision QoS/admission-risk features.
ρ_i^{sla}	Pre-decision edge-SLA-risk component used inside the Stage-2 rejection-aware demand branch.
$\mathbf{d}_i^{\text{raw}}$	Bounded demand-score vector used before Stage-2 demand-branch capacity normalization.
$\mathbf{z}_i$	Allocator input vector containing task, state, policy-output, demand, and pre-decision risk features for task i .
f_ϕ	Residual resource-allocation network parameterized by ϕ .
ζ_i^{term}	Binary terminal-mask indicator for the DDQN target; it is one at split, gap, sequence, or final-sample boundaries where bootstrapping is suppressed.

availability features before learning. Formulas such as $c_i/u_{j,t}^e$ and d_i/B_t^{eff} are therefore benchmark-consistent service-pressure terms, not calibrated processor or radio measurements.

The scheduler chooses a binary action $a_i \in \{0, 1\}$, where 0 denotes edge execution and 1 denotes cloud execution. The binary action space keeps the decision interpretable. Rejection is not an action. It is an outcome that can occur after the selected destination is checked against resource, security, and deadline feasibility.

3.3 Edge Execution Model

If $a_i = 0$, task i is executed at edge node j_i . Its edge latency is modeled as

$$L_i^e = L_i^{e,\text{queue}} + L_i^{e,\text{exec}} + L_i^{e,\text{local}} + L_i^{e,\text{fault}}, \quad (1)$$

where $L_i^{e,\text{queue}}$ increases with q_{j_i,t_i}^e and recent arrivals, $L_i^{e,\text{exec}}$ depends on $c_i/u_{j_i,t_i}^e$, $L_i^{e,\text{local}}$ represents local transfer and broker overhead, and $L_i^{e,\text{fault}}$ captures degradation, isolation, or failure penalties. Edge execution is feasible only when generated resource, node-state, security, battery, corruption, and admission checks pass:

$$\Phi_i^e = \Phi_{i,\text{cpu}}^e \wedge \Phi_{i,\text{mem}}^e \wedge \Phi_{i,\text{state}}^e \wedge \Phi_{i,\text{sec}}^e \wedge \Phi_{i,\text{bat}}^e \wedge \Phi_{i,\text{corr}}^e \wedge \Phi_{i,\text{adm}}^e. \quad (2)$$

Here $\Phi_{i,\text{cpu}}^e$ includes $u_{j_i,t_i}^e \geq u_i^{\min}$, $\Phi_{i,\text{mem}}^e$ includes $m_{j_i,t_i}^e \geq \bar{m}_i$, and $\Phi_{i,\text{state}}^e$ excludes failed or isolated edge states. The remaining terms encode security compatibility, battery or local-device restrictions, corrupt-task flags, and generator-side deadline/admission consistency. The implementation uses lookup tables from the generated CSV state files rather than recomputing each term from a closed-form physical model. Equations (1) and (2) describe the conceptual decomposition used to interpret those computed labels.

3.4 Cloud Execution Model

If $a_i = 1$, the task is sent to the cloud. Its latency is

$$L_i^c = L_i^{\text{up}} + L_t^n + L_i^{c,\text{queue}} + L_i^{c,\text{exec}} + L_i^{c,\text{cold}}, \quad (3)$$

where L_i^{up} depends on task size and uplink bandwidth, L_t^n is network delay, $L_i^{c,\text{queue}}$ is cloud queue delay, $L_i^{c,\text{exec}}$ is cloud processing delay, and $L_i^{c,\text{cold}}$ accounts for cold starts, overload, and maintenance windows. Cloud execution can be attractive for large compute tasks when edge resources are scarce. It is less attractive when the network state is degraded or when the deadline is tight. Cloud feasibility is written analogously:

$$\Phi_i^c = \Phi_{i,\text{cpu}}^c \wedge \Phi_{i,\text{mem}}^c \wedge \Phi_{i,\text{state}}^c \wedge \Phi_{i,\text{net}}^c \wedge \Phi_{i,\text{sec}}^c \wedge \Phi_{i,\text{corr}}^c \wedge \Phi_{i,\text{adm}}^c. \quad (4)$$

The cloud resource terms cover generated cloud CPU and memory availability, $\Phi_{i,\text{state}}^c$ covers maintenance, overload, and cold-start state, and $\Phi_{i,\text{net}}^c$ covers outage, packet-loss, and effective-bandwidth admission. Security, corrupt-task, and deadline/admission consistency are checked on the selected cloud path in the same way they are checked on the selected edge path.

3.5 QoS and Rejection Outcomes

A task satisfies its SLA if the selected latency is no greater than the task-specific deadline:

$$\text{SLA}_i = \mathbb{I}[L_i(a_i) \leq \delta_i], \quad (5)$$

where $L_i(a_i) = L_i^e$ for edge execution and $L_i(a_i) = L_i^c$ for cloud execution. The SLA-miss indicator is

$$M_i = 1 - \text{SLA}_i. \quad (6)$$

The selected path may also fail feasibility or admission checks. The selected-path predicate is

$$\Phi_i(a_i) = \begin{cases} \Phi_i^e, & a_i = 0, \\ \Phi_i^c, & a_i = 1. \end{cases} \quad (7)$$

The failure indicator and reported rejection metric are then

$$F_i = \mathbb{I}[\neg \Phi_i(a_i)], \quad (8)$$

$$R_i = F_i. \quad (9)$$

Thus R_i is not a third scheduler action and is not the same as an SLA miss. It is the feasibility or admission

failure of the path that the policy selected. For discussion that needs a broader QoS-failure indicator, define

$$Q_i = \mathbb{I}[M_i = 1 \vee F_i = 1]. \quad (10)$$

Thus, SLA miss, rejection, and broad QoS failure are related but not identical. M_i records latency-deadline violation, R_i records selected-path infeasibility or admission failure, and Q_i records their union. The policy is not rewarded for choosing rejection because rejection is not an action; it is an outcome of the selected edge or cloud path.

The pre-decision QoS/admission-risk estimate is denoted by $\hat{\rho}_i$. It is different from the realized outcomes M_i , R_i , and Q_i : $\hat{\rho}_i$ is computed before action evaluation from observable risk predictors such as deadline tightness, low battery, corruption flags, local queue pressure, outage/congestion state, and destination availability. It is used for reward shaping and Stage-2 allocation, while M_i and R_i are computed only after the selected edge or cloud path is evaluated. Aggregate latency is computed on valid held-out rows from the selected edge/cloud latency table; SLA miss and rejection are then computed from the selected path and benchmark feasibility flags.

3.6 Latency-Reference Oracle

The Oracle reported in the results is a non-trainable latency reference over the generated edge and cloud alternatives. Let L_{sent} denote the sentinel value used by the label-generation pipeline for invalid or unavailable latency entries. The Oracle action is

$$a_i^{\text{orc}} = \arg \min_{a \in \{0,1\}: L_i(a) < L_{\text{sent}}} L_i(a), \quad L_i^{\text{orc}} = L_i(a_i^{\text{orc}}). \quad (11)$$

This rule bounds the generated latency achievable by a binary edge/cloud decision on the offline sequence. It is not a deployable policy, and it is not a universal ceiling for every QoS metric. Oracle SLA miss and rejection can remain nonzero because the rule minimizes finite generated latency among available alternatives; it does not guarantee that every task is feasible or deadline-satisfying.

3.7 Problem Objective

The offloading objective is to find a policy $\pi(a|x_i)$ that minimizes expected latency and QoS penalties:

$$\min_{\pi} \mathbb{E}_i [\tilde{L}_i(a_i) + \lambda_{\text{sla}} M_i + \lambda_{\text{rej}} R_i + \lambda_{\text{edge}} P_i^e + \lambda_{\text{heavy}} H_i], \quad (12)$$

subject to edge, cloud, and network feasibility constraints. Here x_i is the 30-dimensional feature vector, $\tilde{L}_i(a_i)$ is normalized selected latency, P_i^e is an edge-pressure penalty, and H_i penalizes routing heavy cloud-favorable tasks to edge when that choice is predicted to be harmful. The λ terms control the relative cost of SLA miss, selected-path rejection, overload, and heavy-task mismatch.

If raw latency is used in an implementation, the corresponding λ values are interpreted in latency-equivalent units. The proposed Fed-DDQN learns this decision rule through offline replay over the fixed benchmark

sequence: policy actions determine the reward for the sampled task, but the next task/state is exogenous and precomputed instead of being mutated online by the selected action. The final test evaluation then applies the learned policy to a held-out temporal interval.

4 DATASET AND DATA ENGINEERING

The experiments use the benchmark dataset for repeatable evaluation of edge-cloud IoT offloading under stress. It is a realistic synthetic benchmark generated for this project with temporal workload phases, correlated channel states, edge pressure, cloud load, and fault windows. The paper treats it as the final benchmark dataset for the reported experiments.

The dataset contains 100,000 task rows, 50 edge nodes, 10 cloud nodes, and 1,000 timesteps. The six CSV files describe task records, static edge nodes, per-timestep edge state, per-timestep network state, static cloud nodes, and per-timestep cloud state. Table 4 summarizes the files.

4.1 Temporal Workload Phases

The task generator creates workload phases instead of sampling every row independently. Normal periods are mixed with morning and evening bursts, emergency bursts, firmware rollout windows, industrial telemetry surges, and video or AI-heavy peaks. Multiple tasks can arrive at the same timestep and create pressure on queues, bandwidth, and available compute. The resulting offloading problem is harder than choosing between edge and cloud from isolated task descriptors.

Task size uses a mixture of lognormal and heavy-tailed samples. CPU cycles, memory demand, deadline, priority, security requirements, and battery state are correlated with task type and device type. For example, emergency tasks have tight deadlines, firmware updates have larger transfer sizes but relaxed deadlines, and AI/video tasks tend to have heavier compute and memory requirements. These correlations matter because a model trained on uncorrelated synthetic features can learn shortcuts that do not survive even modest workload realism.

4.2 Network, Edge, and Cloud State

Network state is generated as a time-correlated process. Congestion, outage, and jitter windows affect packet loss, delay, available bandwidth, and SNR for multiple consecutive timesteps, which is closer to operational behavior than independent packet-loss draws at every row. Edge state also carries temporal pressure: local task arrivals increase queue length, reduce effective CPU and memory availability, and drain energy. Failures and degradation can occur in clusters, and some nodes partially recover after a fault window.

Cloud state is also stateful. Maintenance, overload, and cold starts can raise queue and latency for multiple timesteps. The generator does not make cloud unattractive by construction. The benchmark includes edge-favorable, cloud-favorable, and ambiguous cases, since a dataset where one destination is almost always best would not test policy learning.

Table 4: the benchmark dataset files and schema roles.

File	Rows	Columns	Role
dataset_A.csv	100,000	23	Task records, device/task properties, arrival time, assigned edge, dependency and risk flags.
edge_nodes.csv	50	8	Static edge-node capacity, tier, renewable-power flag, and location zone.
edge_state.csv	50,000	10	Per-timestep edge CPU, memory, queue, latency, energy, failure, degradation, and isolation state.
network_state.csv	1,000	11	Per-timestep uplink/downlink bandwidth, delay, packet loss, load, SNR, outage, congestion, and jitter.
cloud_nodes.csv	10	6	Static cloud region, CPU, memory, bandwidth, and SLA tier.
cloud_state.csv	10,000	9	Per-timestep cloud CPU, memory, queue, latency, maintenance, overload, and cold-start state.

Table 5: Reproducibility package components for the reported benchmark.

Component	Role	Release/check status
Dataset generator	Creates the task, edge, network, and cloud CSV files with temporal bursts, correlated channel state, and fault windows.	Project artifact; generator-native parameters documented with the dataset.
Six CSV files	Provide tasks, edge nodes, edge state, network state, cloud nodes, and cloud state for repeatable evaluation.	Row counts and column schemas are checked before training.
Temporal split protocol	Defines train, validation, gap, and held-out test windows by arrival timestep.	Fixed in the manuscript and reused across all policies.
Label-generation logic	Computes edge/cloud latency estimates, SLA flags, and selected-path rejection outcomes.	Kept outside the Stage-1 feature matrix to avoid target leakage.
Analysis workflow and figure exporter	Produce model outputs, tables, and manuscript figures from the same held-out actions.	Reused for final tables and diagnostics.
Checksum manifest	Verifies file identity for the benchmark package and records CSV schema size.	Exported with the manuscript bundle as <code>dataset_checksums.csv</code> .

The channel model used by the feature pipeline can be interpreted through an effective bandwidth term:

$$B_t^{\text{eff}} = \max(\epsilon_B, B_t^{\text{up}}(1 - \ell_t)(1 - o_t)(1 - \xi_t)), \quad (13)$$

where B_t^{up} is uplink bandwidth, $\ell_t \in [0, 1]$ is packet-loss rate, $o_t \in \{0, 1\}$ is an outage indicator, and $\xi_t \in [0, 1]$ is a clipped jitter/congestion discount. The floor $\epsilon_B > 0$ prevents non-positive bandwidth in downstream pressure ratios. The term is not a physical radio simulator. Its role is to expose the learner to interacting bandwidth, loss, outage, and jitter effects instead of treating those quantities as unrelated features.

Queue pressure is also carried over in time. A simplified recurrence is

$$q_{j,t+1}^e = \max(0, q_{j,t}^e + A_{j,t} - \mu_{j,t}^e + \omega_{j,t}), \quad (14)$$

where $A_{j,t}$ is the local arrival count, $\mu_{j,t}^e$ is effective service capacity, and $\omega_{j,t}$ is a signed stress adjustment: degradation or isolation can increase it, while recovery can decrease it. The outer projection preserves the queue invariant $q_{j,t}^e \geq 0$. The CSV state tables store the realized values generated by this logic, while the policy observes their effect through edge queue, CPU, memory, energy, network, and cloud-load features.

4.3 Race-Like Contention Approximation

The offline benchmark reads precomputed CSV state files, so it does not simulate concurrent state mutation while a policy acts. Instead, the benchmark dataset approximates race-like pressure through precomputed contention. For each timestep, the generator counts total arrivals, edge-local arrivals, likely cloud-heavy tasks, and recent workload carryover. These values influence edge queues, edge availability, network quality, and cloud queue state. The learned model sees simultaneous arrivals as different from isolated arrivals, even though the benchmark remains a fixed offline dataset.

4.4 Labels, Features, and Leakage Control

Labels are produced by computing edge and cloud latency with the same lookup tables used by the final implementation. Rejection and SLA flags are derived outcomes. Sentinel latencies are kept out of the feature matrix to avoid polluting training with artificial extreme values. The Stage-1 feature matrix has 30 columns and is checked against `features_raw.shape == (100000, 30)`. The realized rejection label is not included as a Stage-1 input feature.

The label-generation logic can use additional task and state fields to compute edge latency, cloud latency, SLA violation, and rejection outcomes. Those derived outcomes are then kept outside the Stage-1 feature vector. The model can observe operational predictors such as queue, SNR, outage, and cloud overload without directly receiving the target labels used for evaluation.

The temporal split is purged to reduce leakage across adjacent timesteps. Training uses arrivals up to timestep 700. Timesteps 701–715 are ignored as a gap. Validation uses timesteps 716–850. Timesteps 851–865 are ignored as the second gap. The final test split uses timesteps 866–1000. The scaler is fit only on the training rows, and validation/test rows are transformed using that scaler.

The temporal split is stricter than a random train/test split. It asks the policy to generalize forward in time, which is closer to deployment practice for workload traces. It also keeps the test split out of checkpoint selection. Validation is used for early stopping and model choice; the final test interval is used only for reporting.

For reproducibility, the dataset generator, six CSV files, temporal split definition, label-generation logic, analysis workflow, figure exporter, and manuscript figures are maintained as a single project package. The manuscript export bundle now includes a checksum manifest so future users can verify that their local CSV files match the benchmark used for the reported tables.

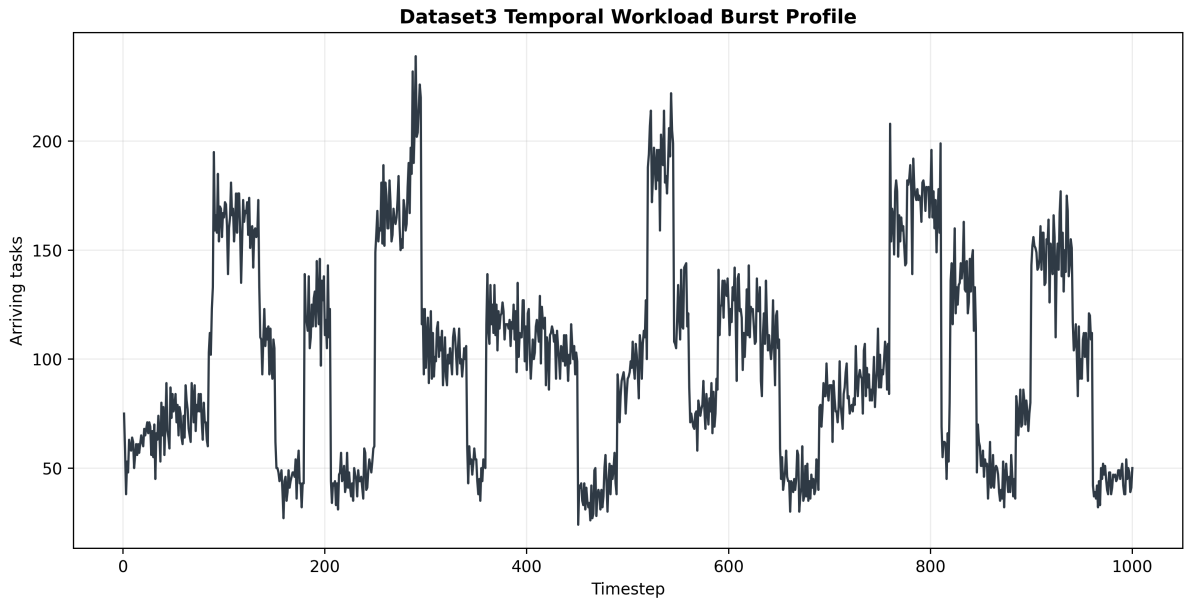


Figure 2: Workload and network timeline in the benchmark dataset, showing burst and stress windows generated for repeatable edge-cloud evaluation.

Table 6: Exact Stage-1 feature list used for offloading model input.

Group	Included Stage-1 feature names
Task descriptors	<code>task_size_mb</code> , <code>cpu_cycles</code> , <code>memory_req_mb</code> , <code>deadline_ms</code> , <code>priority_level</code> , <code>energy_required</code> , <code>security_sensitivity</code> , <code>task_type</code> , <code>device_type</code> .
Task and device flags	<code>is_real_time</code> , <code>is_encrypted</code> , <code>is_low_battery</code> , <code>has_dependency</code> , <code>retransmission_count</code> .
Edge state	<code>e_cpu</code> , <code>e_mem</code> , <code>e_queue</code> , <code>e_energy</code> , <code>e_fail</code> , <code>e_deg</code> .
Network/channel state	<code>n_delay</code> , <code>n_bw</code> , <code>n_loss</code> , <code>n_snr</code> , <code>eff_bw</code> , <code>n_out</code> , <code>n_jit</code> , <code>n_cong</code> .
Cloud state	<code>c_cpu</code> , <code>c_over</code> .
Feature-label boundary	The Stage-1 feature matrix has 30 columns and is checked against <code>features_raw.shape == (100000, 30)</code> . Label-only fields such as <code>rejection_flag</code> , <code>rejected</code> , <code>sla_violated</code> , <code>edge_latency</code> , and <code>cloud_latency</code> are not Stage-1 input features.

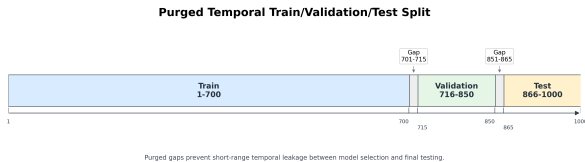


Figure 3: Purged temporal split used for model selection and final testing. Gap windows reduce near-boundary leakage between train, validation, and final test intervals.

4.5 Why the Dataset Changes the Evaluation

In a dataset with mostly independent task rows, a model can perform well by learning a simple association between task size and destination. In the benchmark dataset, the same task can become edge-favorable or cloud-favorable depending on queue carryover, congestion state, outage state, local degradation, and cloud maintenance. This makes the offloading label less static. It also creates more ambiguous samples, which is why the latency-reference Oracle remains noticeably below the learned policy.

The dataset also makes rejection more meaningful. Corrupt tasks, deadline-admission flagged tasks, low-battery cases, and outage windows create situations where the selected action can be infeasible even when the other destination is not perfect. That failure mode is

closer to a deployment scheduler: not every bad outcome is a high latency value; some tasks are rejected because the resource or state condition cannot support execution. The feature pipeline keeps these cases visible in the label and metric logic without letting sentinel latency values dominate model input.

the benchmark dataset functions as a controlled stress benchmark for algorithm comparison. It combines repeatable conditions with burst, fault, queue, and maintenance structure, making it suitable for evaluating offloading and resource-allocation decisions before future measured-trace or online-deployment studies.

5 PROPOSED FEDERATED DDQN OFFLOADING FRAMEWORK

5.1 MDP Definition

The Stage-1 scheduler is formulated as an offline replay MDP over the fixed the benchmark dataset sequence. At each decision step, the agent observes a feature vector $x_i \in \mathbb{R}^{30}$ derived from task, edge, network, and cloud state. It selects action $a_i \in \{0, 1\}$, receives reward r_i , and then advances to the next sampled task state. The next state is exogenous in the benchmark: selecting edge or cloud changes the reward assigned to the current task, but it does not mutate the CSV queue state for the following timestep during training. The state includes task demand and urgency, local edge availability, queue

pressure, channel indicators, and cloud load indicators. It does not include artificial sentinel values for rejected tasks.

The action space is intentionally binary. Edge execution is selected by $a_i = 0$ and cloud execution by $a_i = 1$. A rejected task is a failure outcome after feasibility checks, not a choice the policy can make. This makes the comparison stricter. The policy must decide where to execute the task, and it is penalized if that destination leads to infeasibility or SLA failure.

5.2 DDQN and Dueling Q-Network

The proposed policy uses a Double DQN target:

$$y_i = r_i + \gamma (1 - \zeta_i^{\text{term}}) Q_{\theta^-} \left(x_{i+1}, \arg \max_{a'} Q_{\theta}(x_{i+1}, a') \right), \quad (15)$$

where θ are the online network parameters, θ^- are the target network parameters, and ζ_i^{term} is a binary terminal-mask indicator. The mask is one at sequence ends, purged split or gap boundaries, and final-sample boundaries where bootstrapping should not cross into another evaluation interval. The loss is the Huber temporal-difference loss between $Q_{\theta}(x_i, a_i)$ and y_i . In the present offline benchmark, the next state remains exogenous and precomputed; the terminal mask defines where value bootstrapping is conceptually suppressed, not where an online simulator resets because of the selected action.

The Q-network uses a dueling architecture:

$$Q(x, a) = V(x) + A(x, a) - \frac{1}{|A|} \sum_{a' \in A} A(x, a'). \quad (16)$$

The value stream estimates the quality of the state, while the advantage stream estimates the relative value of edge versus cloud. This separation helps when task urgency or network degradation dominates the decision regardless of the final action.

5.3 Reward Design

The reward follows the final evaluation metrics. A simplified form is

$$r_i = -\alpha_L \tilde{L}_i - \alpha_S M_i - \alpha_R R_i - \alpha_E P_i^e - \alpha_H H_i, \quad (17)$$

where $\tilde{L}_i$ is normalized selected latency, M_i indicates SLA miss, $R_i = F_i$ indicates selected-path rejection or admission failure, P_i^e is edge pressure or edge-overuse cost, and H_i penalizes routing heavy AI, video, image, or firmware-like tasks to edge when cloud is clearly better. The reward does not force a permanent preference for edge or cloud. It makes a poor destination costly for the reason it is poor: latency, SLA miss, rejection, overload, or heavy-task mismatch.

Because R_i is defined as feasibility/admission failure rather than $M_i \vee F_i$, an SLA miss and a rejection are not automatically double-counted. A task receives both penalties only when the selected path both misses its deadline and fails the feasibility/admission predicate.

For reward shaping, L_i^{best} and L_i^{worst} denote the minimum and maximum finite generated latency over the edge and cloud alternatives after infeasible sentinel values are excluded. The regret term uses the same normalized selected-latency scale as $\tilde{L}_i$.

The reward design controls the edge bias that can appear when a learner overvalues local execution for tight-deadline tasks. Prioritized replay [20] and stronger action-aware pressure are diagnostic variants, but the proposed configuration uses standard replay, light FedProx regularization, an explicit validation penalty for excessive edge usage, and a heavy-task penalty. These choices keep the model within the Fed-DDQN family while discouraging the policy from routing large cloud-favorable tasks to constrained edge nodes.

5.4 Channel-Aware Decision Margin

The binary action is simple, but the decision boundary is not. The learned Q-values can be read through a decision margin

$$\Delta Q_i = Q_{\theta}(x_i, \text{edge}) - Q_{\theta}(x_i, \text{cloud}). \quad (18)$$

The policy selects edge when $\Delta Q_i > 0$ and cloud otherwise. The margin connects the neural decision to the physical factors in the state vector. A compact hand-written proxy for the same intuition separates edge-side and cloud-side penalties on a normalized score scale:

$$\hat{S}_i^e = \tilde{L}_i^e + \beta_q \tilde{q}_{j_i, t_i}^e + \beta_h h_i, \quad (19)$$

$$\hat{S}_i^c = \tilde{L}_i^c + \beta_\ell \ell_{t_i} + \beta_o o_{t_i} + \beta_\xi \xi_{t_i}, \quad (20)$$

$$\Delta S_i = \hat{S}_i^e - \hat{S}_i^c. \quad (21)$$

Here $\tilde{L}_i^e$ and $\tilde{L}_i^c$ are normalized generated latency terms, $\tilde{q}_{j_i, t_i}^e$ is normalized local queue pressure, and the β coefficients are dimensionless interpretive weights. The terms h_i , ℓ_{t_i} , o_{t_i} , and ξ_{t_i} represent heavy-task mismatch, packet loss, outage, and jitter/congestion stress. This proxy is illustrative only; reported decisions are produced by the learned Q-network, not by this hand-written score. It describes the kind of nonlinear boundary the network must learn: edge is attractive when the adjusted cloud path is more expensive, but that advantage should shrink when local queue pressure or heavy-task mismatch raises the edge-side cost. For this reason, the state contains both task descriptors and channel/resource indicators, and the final evaluation reports action distribution and scenario behavior instead of relying only on average latency.

5.5 Zone-Based Federated Training

The dataset is partitioned into edge zones. Each zone trains a local DDQN model on its own non-identical distribution. After local updates, the server aggregates zone models using weighted FedAvg:

$$\theta^{(r+1)} = \sum_{z=1}^Z \frac{n_z}{\sum_{k=1}^Z n_k} \theta_z^{(r+1)}, \quad (22)$$

where n_z is the number of samples used by zone z . This weighting prevents small zones from dominating the global update while still allowing their local behavior to influence the shared model.

FedProx regularization is used to reduce excessive local drift:

$$\mathcal{L}_z = \mathcal{L}_{\text{TD}, z} + \frac{\mu}{2} \|\theta_z - \theta_g\|_2^2. \quad (23)$$

Table 7: Reward components used to align Fed-DDQN training with final QoS evaluation.

Term	Meaning	Intended behavior
$-\alpha_L \tilde{L}_i$	Normalized selected latency penalty.	Prefer lower-latency edge or cloud decisions.
$-\alpha_S M_i$	SLA miss penalty.	Discourage choices that violate task-specific deadlines.
$-\alpha_R R_i$	Rejection/admission penalty.	Penalize selected-path resource, security, battery, corruption, or destination-state infeasibility.
$-\alpha_E P_i^e$	Edge pressure or edge-overuse penalty.	Prevent the policy from routing too much work to constrained edge nodes.
$-\alpha_H H_i$	Heavy-task misrouting penalty for AI, video, image, and firmware-like tasks.	Shift large or cloud-favorable workloads away from edge when local execution is not actually better.

Table 8: Reward and model-selection details used for reproducibility.

Component	Implemented form	Purpose	Status
Latency shaping	$1.25\{1 - 2(L_i - L_i^{\text{best}})/(L_i^{\text{worst}} - L_i^{\text{best}} + \epsilon)\}$	Rewards lower selected latency relative to the edge/cloud alternatives.	Fixed
Regret penalty	$-0.75 \text{ clip}(L_i - L_i^{\text{best}}, 0, 2)$ after normalization	Penalizes choosing a destination far from the better generated alternative.	Fixed
SLA margin	Positive margin weight 1.05; negative margin weight 2.10	Makes deadline misses more costly than equal-sized positive slack is beneficial.	Fixed
Rejection/admission	-2.25 when selected path is infeasible or a deadline-admission flag applies	Aligns training with selected-path rejection reporting.	Fixed
Heavy-task edge misuse	Penalty for heavy AI, video, image, and firmware-like tasks routed to edge when cloud is not worse	Reduces edge bias for cloud-favorable heavy workloads.	Fixed
Energy and battery shaping	Small energy requirement penalty and low-battery non-urgent penalty	Discourages avoidable edge use under device pressure.	Fixed
Reward clipping	Final reward clipped to $[-5, 5]$	Prevents rare generated extremes from dominating replay updates.	Fixed
Validation score	$\bar{L} + 5.0M_{\text{SLA}} + 2.5R + 0.8[U^e - 78]_+ + 1.2H$	Selects the checkpoint by latency, SLA, rejection, edge balance, and heavy-task behavior.	Validation-selected

Algorithm 1: Zone-Aware Federated DDQN Training

Input: train split, zone partitions, global network Q_θ , target network Q_{θ^-} , replay buffers, maximum rounds.

Output: validation-selected global policy.

- 1) Initialize the global dueling DDQN and broadcast its weights to all active edge-zone clients.
- 2) For each federated round, sample local training transitions within each zone and update the local online network with the terminal-masked DDQN target in (15).
- 3) Add the FedProx drift penalty in (23) using the current global model as the anchor.
- 4) Clip or sanitize unstable drift values before the optimizer step, matching the final numerically stable training path.
- 5) Return local model weights and sample counts from each zone.
- 6) Aggregate local models by weighted FedAvg as in (22).
- 7) Evaluate the global model on the validation interval. Save the best checkpoint by the validation score in (24).
- 8) Stop when patience is reached or the maximum number of rounds is exhausted. Restore the best global checkpoint before test evaluation.

Figure 4: Training procedure for the proposed Fed-DDQN policy. The test split is not used inside the training loop.

The final implementation uses $\mu = 0.003$ with a drift clamp to avoid numerically unstable local-drift values. The proximal term is light enough to allow local adaptation but strong enough to discourage zone models from moving too far away from the current global policy.

5.6 Validation Checkpointing

Model selection uses the validation interval, not the test interval. The validation score is

$$S_{\text{val}} = \bar{L}_{\text{val}} + 5.0M_{\text{SLA},\text{val}} + 2.5R_{\text{val}} + 0.8[U_{\text{val}}^e - 78]_+ + 1.2H_{\text{val}}. \quad (24)$$

where $\bar{L}_{\text{val}}$ is average validation latency in milliseconds, $M_{\text{SLA},\text{val}}$ is SLA miss percentage, R_{val} is rejection percentage, U_{val}^e is validation edge usage percentage, and H_{val} is the percentage of heavy cloud-favorable

tasks routed to edge. The operator $[x]_+ = \max(x, 0)$ applies the edge-overuse penalty only above the 78% soft cap. Percentages are expressed as percentage points in $[0, 100]$, so the coefficients have units of milliseconds per percentage point. The best global model is restored before final test evaluation. The final training run selected validation round 7 with score 131.2317, and training ran for 13 rounds before stopping.

5.7 Learning Dynamics and Complexity

The communication cost of one federated round is proportional to the number of participating zones and the number of model parameters:

$$\mathcal{B}_r \approx 2Z|\theta|b, \quad (25)$$

Algorithm 2: Validation Checkpointing and Test-Only Evaluation

Input: trained round checkpoints, validation split, untouched test split, latency/rejection lookup functions.
Output: final comparison table and recorded policy actions.

- 1) For each candidate global model, infer validation actions with `torch.inference_mode()`.
- 2) Compute selected latency, SLA miss percentage, and rejection percentage on validation rows only.
- 3) Rank checkpoints with $S_{\text{val}} = \bar{L}_{\text{val}} + 5M_{\text{SLA, val}} + 2.5R_{\text{val}} + 0.8[U_{\text{val}}^e - 78]_+ + 1.2H_{\text{val}}$, where miss, rejection, edge-use, and heavy-task misuse rates are measured in percentage points.
- 4) Restore the checkpoint with the lowest validation score.
- 5) Freeze the model and infer actions on the test split once. Record those actions with a dataset/configuration signature.
- 6) Compute all final metrics from the test split only.
- 7) Reuse the recorded actions for plots, task-type analysis, scenario tables, and allocator input so repeated inference does not change reported results.

Figure 5: Checkpointing and evaluation discipline used to keep the final test interval out of model selection.

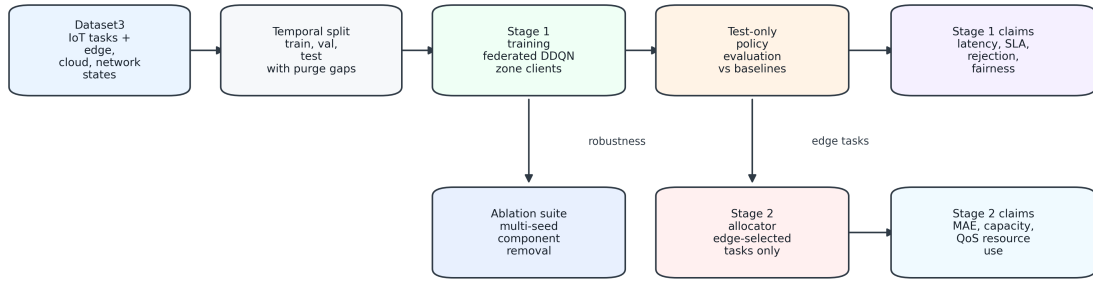
Evaluation Methodology for the Proposed Fed-DDQN + Adaptive Allocation System

Figure 6: Experimental flow from label generation through purged temporal splitting, federated training, validation checkpointing, test-only comparison, and Stage-2 allocation.

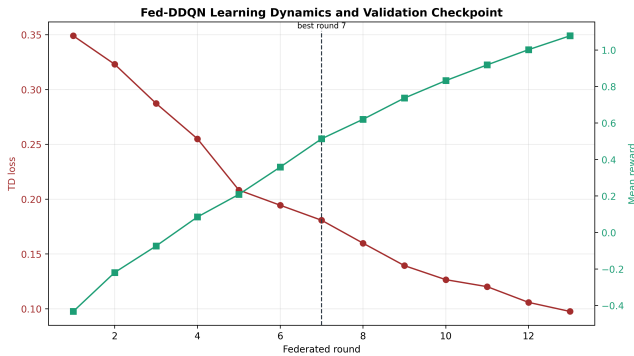


Figure 7: Learning dynamics for the proposed federated policy. The selected checkpoint is based on validation score rather than final training loss alone.

where b is the number of bytes per parameter and the factor of two accounts for download and upload. This expression measures model-transfer volume per round, not a universal byte-saving claim. Federated training avoids pooling raw zone records, but the total bytes can exceed one-time raw-feature transfer depending on Z , $|\theta|$, the number of rounds, parameter precision, compression, and secure-aggregation overhead.

The training design is not only about decentralization. It lets the model learn from zone-specific behavior without assuming that every region has the same task mix or channel profile. This motivates the federated DDQN formulation instead of a centralized DDQN with a larger feature vector.

5.8 Design Rationale

The final design follows a simple rule: keep the reinforcement-learning action space focused on the edge/cloud choice, and push resource assignment into the second stage. Adding more mechanisms to Stage 1 does not automatically improve the complete objective. Action-aware pressure and prioritized replay can make training more aggressive, but they also change the latency–SLA–rejection balance. The proposed configuration keeps the Fed-DDQN core and uses a conservative training recipe selected by validation performance.

Three implementation choices matter for reproducibility. First, the environment is evaluation-aligned: the reward is computed against the same latency and rejection logic used in final comparison. Second, the validation score combines latency, SLA miss, and rejection instead of choosing the lowest training loss.

Third, the test comparison is batched after policy inference, so plots and scenario tables reuse the same actions. These details make the final numbers easier to reproduce and audit.

The design also avoids a tempting shortcut. One could add a rejection action and let the policy reject difficult tasks directly. That would simplify training, but it would make the reported rejection rate partly a policy preference rather than an outcome of infeasibility. The implementation keeps the harder binary action setting: if a task fails, the failure is counted against the selected edge or cloud decision.

6 REJECTION-AWARE ADAPTIVE RESOURCE ALLOCATION

6.1 Role of the Second Stage

The second stage is applied only after Fed-DDQN selects edge execution. It does not change the edge/cloud decision. Its role is to assign CPU, memory, and bandwidth shares for edge-selected tasks while respecting demand, priority, deadline pressure, and pre-decision QoS/admission risk. This separation keeps the primary contribution focused on task offloading while moving the pipeline closer to an edge resource manager.

Let $\mathcal{I}_e = \{i : a_i = 0\}$ denote the set of tasks selected for edge execution. The reported Stage-2 metrics use the filtered allocator-valid set

$$\mathcal{I}_e^{\text{valid}} = \{i \in \mathcal{I}_e : \psi_i^{\text{alloc}} = 1\},$$

where ψ_i^{alloc} marks tasks with complete proxy-target and edge-capacity fields. In the test interval this gives 6,563 edge-selected allocator-valid tasks out of 9,733 allocator-valid test tasks with complete resource targets and edge-capacity metadata. This denominator is narrower than the full offloading test split because allocator evaluation requires complete generated proxy-target and edge-capacity fields. For each task, the allocator predicts

$$\hat{\mathbf{y}}_i = [\hat{y}_i^{\text{cpu}}, \hat{y}_i^{\text{mem}}, \hat{y}_i^{\text{bw}}], \quad (26)$$

where the three components represent normalized CPU, memory, and bandwidth assignment. For group-wise normalization, define the active allocator group

$$\mathcal{G}_e^{\text{valid}}(j, t) = \{\ell \in \mathcal{I}_e^{\text{valid}} : j_\ell = j, t_\ell = t\}. \quad (27)$$

This keeps the capacity-normalization denominator tied to the same allocator-valid edge-selected set used for evaluation.

6.2 Demand-Proportional Base Allocation

The allocator begins with a demand-proportional base vector $\mathbf{b}_i$. This base allocation is a reasonable first estimate because larger tasks generally need more compute and memory. Demand alone is not enough, however. A high-priority emergency task with a tight deadline can need a different allocation than a low-priority task with similar size. A task with higher QoS/admission risk may also require a conservative correction to avoid under-allocation.

The base vector can be written component-wise as

$$\begin{aligned} b_i^{\text{cpu}} &= \frac{c_i}{\sum_{\ell \in \mathcal{G}_e^{\text{valid}}(j_i, t_i)} c_\ell}, \\ b_i^{\text{mem}} &= \frac{m_i}{\sum_{\ell \in \mathcal{G}_e^{\text{valid}}(j_i, t_i)} m_\ell}, \\ b_i^{\text{bw}} &= \frac{d_i}{\sum_{\ell \in \mathcal{G}_e^{\text{valid}}(j_i, t_i)} d_\ell}. \end{aligned} \quad (28)$$

where $\mathcal{G}_e^{\text{valid}}(j_i, t_i)$ is the allocator-valid active group competing for edge node j_i at time t_i . The implementation uses normalized feature inputs and generated proxy targets, while the equation gives the allocation intuition.

6.3 Generated QoS-Aware Allocation Targets

The allocator targets are generated proxy targets, not measured optimal allocations from a deployed cluster. Let

$$\mathbf{y}_i^* = [y_i^{*,\text{cpu}}, y_i^{*,\text{mem}}, y_i^{*,\text{bw}}] \quad (29)$$

denote the generated CPU, memory, and bandwidth proxy target. These targets are built from observable task and state fields so that training reflects demand, urgency, local contention, channel stress, and reliability risk without using the final allocation prediction as its own label. The deadline term is converted into a tightness score before it enters the allocator:

$$\bar{\delta}_i = \text{clip}_{[0,1]} \left(\frac{\delta_{\max} - \delta_i}{\delta_{\max} - \delta_{\min} + \epsilon} \right), \quad (30)$$

so larger $\bar{\delta}_i$ means a tighter deadline rather than a longer one. A compact version of the urgency score is

$$u_i^{\text{urg}} = \text{clip}_{[0,1]} (0.30\bar{p}_i + 0.45\bar{\delta}_i + 0.35\chi_i^{\text{rt}} + 0.45e_i^{\text{emg}}), \quad (31)$$

where $\bar{p}_i$ is normalized priority, $\bar{\delta}_i$ is the deadline-tightness score from Eq. (30), χ_i^{rt} is the real-time flag, and e_i^{emg} is the emergency-task flag. Local queue and network stress are summarized as

$$s_i^q = \text{clip}_{[0,1]} (q_{j_i, t_i}^e / 80), \quad (32)$$

$$s_i^n = \text{clip}_{[0,1]} (0.45g_t + 0.55v_t + 0.80o_t + 2\ell_t), \quad (33)$$

where g_t , v_t , o_t , and ℓ_t denote congestion, jitter-storm, outage, and packet-loss indicators. The generated target increases resource shares for urgent, high-priority, reliability-sensitive, low-battery, dependency-heavy, and network-stressed cases, while keeping all three channels clipped to $[0.01, 1.0]$. For example,

$$\begin{aligned} y_i^{*,\text{cpu}} &= \text{clip}_{[0.01,1]} \left(\frac{c_i}{\max(u_{j_i, t_i}^e, \epsilon)} \psi_{\kappa_i}^{\text{cpu}} \right. \\ &\quad \left. \times (1 + 0.35u_i^{\text{urg}})(1 + 0.25s_i^q) \right). \end{aligned} \quad (34)$$

with analogous memory and bandwidth components in $\mathbf{y}_i^*$ using $m_i/m_{j_i, t_i}^e$ and $d_i/B_{t_i}^{\text{eff}}$. The task-type multiplier $\psi_{\kappa_i}^r$ raises CPU and memory for AI/video/image/firmware-like tasks and raises bandwidth for emergency or voice-like tasks. These proxy targets are meant to produce a consistent QoS-aware allocator benchmark; they are not presented as globally optimal resource labels.

6.4 Residual Correction, Risk Projection, and Capacity Normalization

The residual allocator predicts a correction instead of an allocation from scratch:

$$\tilde{\mathbf{y}}_i = \mathbf{b}_i + f_\phi(z_i), \quad (35)$$

where z_i contains task, state, offloading, and risk features, and f_ϕ is a neural network. The correction learns when base demand should be raised or lowered because of deadline, priority, QoS/admission risk, or local pressure.

The risk estimate $\hat{\rho}_i$ is computed before observing the realized outcome. It is a QoS/admission-risk score that combines normalized deadline tightness, priority, real-time/emergency flags, low-battery and corruption indicators, local edge pressure, and network or node-state risk into a bounded value in $[0, 1]$. It excludes realized labels M_i , R_i , and Q_i , Oracle actions, and the generated proxy target $\mathbf{y}_i^*$ from the allocator input. Pre-decision latency estimates may be used as predictors, but realized selected-path outcomes are not fed to the allocator.

The implemented risk projection uses two branches. The residual branch first clips and capacity-normalizes the neural residual output,

$$\mathbf{y}_i^{\text{res}} = \Pi_{\text{cap}}\left(\text{clip}_{[0,1]}(\tilde{\mathbf{y}}_i)\right). \quad (36)$$

The demand branch starts from bounded demand scores $\mathbf{d}_i^{\text{raw}}$ and raises support for urgent or high-risk tasks using

$$\omega_i = \text{clip}_{[0.5,3.0]}(1 + 0.55\rho_i^{\text{sla}} + 0.35u_i^{\text{urg}}), \quad (37)$$

where ρ_i^{sla} is the pre-decision edge-SLA-risk component and u_i^{urg} is the urgency score defined in Eq. (31). The rejection-aware demand branch is

$$\mathbf{y}_i^{\text{dem}} = \Pi_{\text{cap}}(\omega_i \mathbf{d}_i^{\text{raw}}). \quad (38)$$

The final risk-projected allocation then blends the two capacity-safe branches and normalizes again:

$$\mathbf{y}_i^{\text{risk}} = \Pi_{\text{cap}}(0.10\mathbf{y}_i^{\text{res}} + 0.90\mathbf{y}_i^{\text{dem}}). \quad (39)$$

Thus deadline and priority enter through the urgency and QoS/admission-risk features rather than disappearing from the implementation path. Capacity normalization then projects the allocation back into feasible edge capacity:

$$\hat{\mathbf{y}}_i = \Pi_{\text{cap}}(\mathbf{y}_i^{\text{risk}}), \quad (40)$$

so the final assignments do not exceed normalized node-level limits. Without this projection, a raw neural output can have low average error and still violate capacity on many tasks.

For each resource channel $r \in \{\text{cpu}, \text{mem}, \text{bw}\}$ and each allocator-valid active edge group $\mathcal{G}_e^{\text{valid}}(j, t)$, the normalization can be written as

$$\hat{y}_{i,r} = y_{i,r}^{\text{risk}} \min\left(1, \frac{C_{j,t}^r}{\sum_{\ell \in \mathcal{G}_e^{\text{valid}}(j,t)} y_{\ell,r}^{\text{risk}} + \epsilon}\right), \quad (41)$$

where $C_{j,t}^r$ is normalized available capacity and ϵ avoids division by zero. The implementation applies the equivalent scaling operation group-wise, so capacity feasibility is enforced after the neural residual and risk correction.

6.5 Allocator Metrics

The allocator is evaluated with direct proxy-target error, under-allocation, waste, capacity violation, estimated SLA risk, and an efficiency score on $\mathcal{I}_e^{\text{valid}}$. For generated proxy target $\mathbf{y}_i^*$ and predicted final allocation $\hat{\mathbf{y}}_i$, the overall mean absolute error is

$$\text{MAE} = \frac{1}{3|\mathcal{I}_e^{\text{valid}}|} \sum_{i \in \mathcal{I}_e^{\text{valid}}} \|\hat{\mathbf{y}}_i - \mathbf{y}_i^*\|_1. \quad (42)$$

Under-allocation measures how often the predicted allocation falls below the generated proxy target, while waste measures excess assignment beyond that target. Capacity violation measures whether the combined prediction exceeds normalized node capacity after projection.

The reported allocator result is multi-objective. A method with the lowest proxy-target error may still be undesirable if it produces more under-allocation or capacity violation. In the allocator table, the residual plus risk-projection variant gives the best proxy-target efficiency score while keeping capacity violation at zero.

6.6 Why the Allocator is Kept Separate

The allocator could be merged into the offloading policy by expanding the action space to include destination and resource levels. That would be a reasonable future direction, but it would make the current research question less clear. The central question here is whether federated DDQN can learn the edge/cloud decision under channel and QoS stress. If resource allocation were part of the same action, a poor routing decision might be hidden by an allocation correction, or a good routing decision might be penalized because the allocation head was undertrained.

The two-stage structure makes the evaluation easier to read. Stage 1 answers where the task should execute. Stage 2 answers how much edge resource should be assigned after the edge has been chosen. This also matches a common system architecture: a broker selects the execution tier, and a local resource manager performs admission and allocation. The residual allocator is presented as a resource-management companion to the proposed offloading policy, not as a standalone scheduler.

7 EXPERIMENTAL SETUP

All reported numbers are drawn from the same experimental run and are maintained through shared result macros. This keeps tables, figures, and text synchronized across the manuscript.

7.1 Split Protocol and Preprocessing

The temporal split is train arrival time ≤ 700 , validation 716–850, and test 866–1000, with gaps 701–715 and 851–865 excluded. The scaler is fit on the training split only. Final comparisons use only the test split, which prevents

Algorithm 3: Rejection-Aware Residual Resource Allocation

Input: edge-selected tasks $\mathcal{I}_e$, task/resource features, demand-proportional base allocation $\mathbf{b}_i$, trained residual network f_ϕ .

Output: capacity-feasible allocation $\hat{\mathbf{y}}_i$ for CPU, memory, and bandwidth.

- 1) For each edge-selected task, compute the demand-proportional base allocation $\mathbf{b}_i$.
- 2) Concatenate task descriptors, edge state, network state, policy outputs, and the bounded pre-decision QoS/admission-risk estimate $\hat{\rho}_i$ into allocator input z_i ; do not include realized labels M_i , R_i , Q_i , Oracle actions, or generated final allocation targets.
- 3) Predict the residual correction $f_\phi(z_i)$ and form $\tilde{\mathbf{y}}_i = \mathbf{b}_i + f_\phi(z_i)$.
- 4) Build the rejection-aware demand branch from urgency, deadline, priority, edge-SLA risk, and related pre-decision risk features; blend it with the capacity-normalized residual branch.
- 5) Project the result into valid normalized capacity so CPU, memory, and bandwidth assignments remain feasible.
- 6) Report MAE, under-allocation, waste, capacity violation, estimated SLA risk, and efficiency.

Figure 8: Stage-2 allocation procedure for tasks already routed to edge execution by Fed-DDQN.

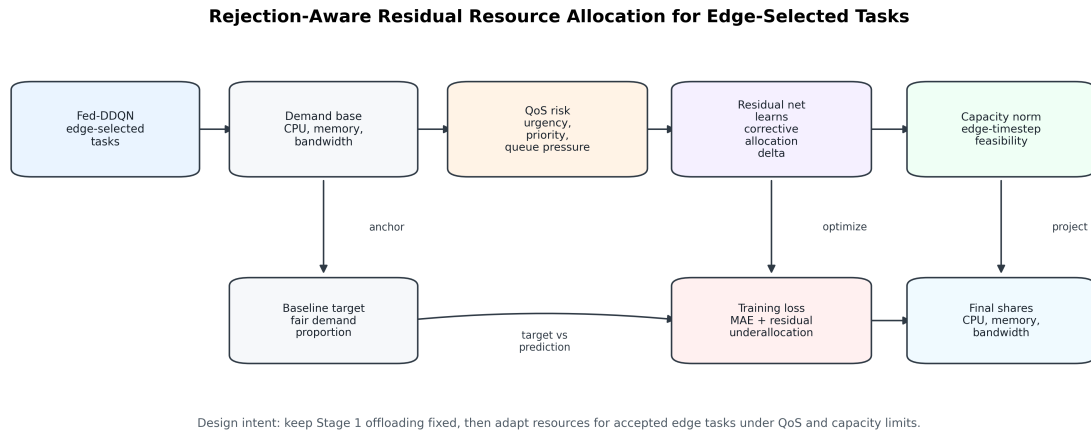


Figure 9: Rejection-aware residual allocation. A demand-proportional base allocation is corrected by a neural residual, adjusted for risk, and normalized to capacity before evaluation.

Table 9: Key training and evaluation configuration.

Item	Value
Dataset	the benchmark dataset.
Task count / edge nodes / cloud nodes / timesteps	100,000 / 50 / 10 / 1,000.
Feature dimension	30 for Stage-1 offloading.
Temporal split	Train ≤ 700 , gap 701–715, validation 716–850, gap 851–865, test 866–1000.
Train / validation / test rows	69,049 / 17,056 / 11,957.
Stage-2 allocator denominator	6,563 edge-selected tasks out of 9,733 allocator-valid test tasks.
Action space	Binary: edge or cloud.
Q-network	Dueling network: 30-256-256 shared extractor, 256-128-1 value stream, 256-128-2 advantage stream.
Discount factor	$\gamma = 0.95$.
Batch size	512.
Federated rounds	15 maximum; proposed run stopped after 13 rounds.
MAX_STEPS	6000.
Target update	Soft update with $\tau = 0.012$; baseline hard update interval 200 where used.
FedProx	$\mu = 0.003$, interval 4, maximum drift clamp 1.0×10^4 .
Replay buffer	Tensor replay buffer; 30,000 per zone for proposed Fed-DDQN.
Optimizer	AdamW for proposed zone models, learning rate 5×10^{-4} , weight decay 1×10^{-4} .
Validation patience	6 rounds.
Verified multi-seed set	42, 77, 123.
Allocator	ResidualResourceAllocator, input dimension 57, hidden widths 160, 96, 48, output dimension 3.
Allocator training	35 epochs, AdamW learning rate 8×10^{-4} , weight decay 8×10^{-5} .

the model from being tuned on the same time interval used for the final claims.

7.2 Comparison Methods

The compared policies are DDQN, MTOSA, FL-DDPG, GTPSO, PTS-RA, JTOS, Fed-DDQN (Proposed), and Oracle. DDQN is implemented as a faithful centralized DDQN-style baseline. FL-DDPG is used as a federated

Table 10: Baseline provenance and local adaptation in the common evaluation pipeline.

Method	Source family	Original objective	Local adaptation and tuning protocol	Fidelity
DDQN	DQN/DDQN/dueling value learning [9–11]	Discrete action-value control with target-network stabilization.	Centralized binary edge/cloud policy trained on the same 30 features, purged split, validation checkpointing, and test-only metrics.	Faithful DDQN-style baseline.
MTOSA	Classical MEC offloading and scheduling family [1–3]	Task placement under latency, computation, and communication constraints.	Implemented as a deterministic scheduling comparator using the same latency, rejection, and scenario scoring functions.	Style-inspired.
FL-DDPG	DDPG actor-critic plus federated MEC learning family [12, 16–18]	Continuous-control actor-critic learning under distributed data.	Adapted as a federated actor-critic comparator inside the same train/validation/test protocol and binary destination evaluation.	Style-inspired comparator.
GTPSO	Population/metaheuristic offloading family [2]	Search over destination choices using swarm-like candidate updates.	Implemented as a PSO-inspired policy comparator scored with the shared held-out evaluation functions.	Style-inspired.
PTS-RA	Priority and resource-aware scheduling family [4, 5]	Allocate computation and communication resources under QoS and energy constraints.	Implemented as a priority/resource heuristic over the same task, edge, channel, and cloud state features.	Style-inspired.
JTOS	Joint task-offloading and scheduling family [6–8]	Joint destination and scheduling decisions for MEC task streams.	Adapted to the binary destination setting while preserving the common split, feature, and metric definitions.	Style-inspired.
Fed-DDQN	Proposed federated value-learning design [10, 11, 13–15]	Zone-aware binary offloading with QoS-aligned reward and validation-selected checkpointing.	Main trainable contribution; validation selects the checkpoint, and the final test interval is used only once for reporting.	Proposed method.
Oracle	Generated latency-reference decision rule	Non-trainable rule that selects the lower finite generated latency among edge and cloud alternatives.	Used only to estimate remaining latency headroom; not available to deployable policies and not a universal QoS ceiling.	Latency reference only.

Table 11: Evaluation metrics reported for Stage-1 offloading and Stage-2 allocation.

Metric	Definition	Used for
Average latency	Mean selected edge/cloud latency over valid test tasks.	Main Stage-1 comparison.
SLA satisfaction	Percentage of tasks with selected latency below the task-specific threshold.	QoS comparison and validation.
Rejection rate	Percentage of selected paths that fail feasibility or admission checks.	Rejection-aware robustness.
Edge usage	Percentage of valid tasks routed to edge execution.	Detecting edge bias or cloud bias.
Communication delay	Mean communication-related delay of selected routes.	Network-sensitive comparison.
Bandwidth use	Mean generator-native normalized bandwidth allocation/usage estimate.	Resource pressure comparison.
Oracle gap	Difference between learned policy latency and latency-reference Oracle latency.	Remaining generated-latency headroom.
Allocator MAE	Mean absolute CPU, memory, and bandwidth error against generated proxy targets.	Stage-2 proxy-target allocation fit on allocator-valid edge-selected tasks.
Under-allocation and waste	Percent under target and percent excess allocation.	Stage-2 QoS/resource balance.
Capacity violation	Percentage of assignments exceeding normalized edge capacity.	Feasibility of allocator output.

actor-critic-style comparator, while MTOSA, GTPSO, PTS-RA, and JTOS are implemented as style-inspired scheduling or metaheuristic comparators inside the same evaluation pipeline. Oracle is included only as a latency-reference rule using generated edge/cloud latency information and is not a deployable trainable method.

7.3 Evaluation Metrics

The main policy metrics are average latency, SLA satisfaction, SLA miss, rejection rate, edge usage, communication delay, and bandwidth use. Scenario and task-type results check behavior under emergency, firmware, corrupt, low-battery, deadline-admission flag, dependency, encryption, and real-time conditions. Multi-seed analysis checks stability across seeds 42, 77, and 123. Ablation analysis tests prioritized replay, action-aware pressure, FedProx, target-update style, and the heavy-task penalty.

7.4 Fairness and Reproducibility Controls

The final comparison is test-only. Validation is used for checkpoint selection and early stopping. The test

interval is not used to choose hyperparameters. All policies are evaluated on the same held-out interval with the same metric functions. The checkpointing algorithm uses the same validation objective as Eq. (24); no simplified validation objective is used for the reported model selection.

The experiments are configured as a controlled benchmark comparison. All methods share the same data, temporal split, feature pipeline, and metric functions. The baseline implementations provide aligned comparators within this evaluation environment, allowing the reported differences to reflect policy behavior rather than differences in preprocessing or scoring.

8 RESULTS AND DISCUSSION

8.1 Main Test Comparison

Table 12 gives the test-only comparison. The proposed Fed-DDQN achieved 136.009 ms average latency, 97.73% SLA satisfaction, 3.68% rejection, and 67.43% edge usage. It reports the lowest latency among the main comparison methods in the table. Relative to

Table 12: Untouched temporal test comparison.

Method	Avg. latency (ms)	SLA (%)	SLA miss (%)	Rejection (%)	Edge usage (%)	Comm. delay (ms)	Bandwidth use (norm.)
DDQN	144.926	97.49	2.51	3.90	73.71	36.642	19.83
MTOSA	231.841	89.71	10.29	11.52	25.50	71.153	51.35
FL-DDPG	138.373	97.67	2.33	3.74	71.23	38.146	22.33
GTPSO	203.551	90.36	9.64	10.89	10.62	68.149	47.39
PTS-RA	228.897	90.61	9.39	10.64	12.49	72.627	50.92
JTOS	207.521	90.54	9.46	10.71	18.30	71.970	54.46
Fed-DDQN (Proposed)	136.009	97.73	2.27	3.68	67.43	40.965	26.76
Oracle	123.310	97.99	2.01	3.43	69.27	39.037	24.92

Note: Bold marks the best deployable/trainable method among the main comparison methods. Oracle is a non-trainable latency reference over generated finite edge/cloud alternatives, not a universal QoS ceiling. Bandwidth use is reported in generator-native normalized units used by the benchmark feature and metric pipeline.

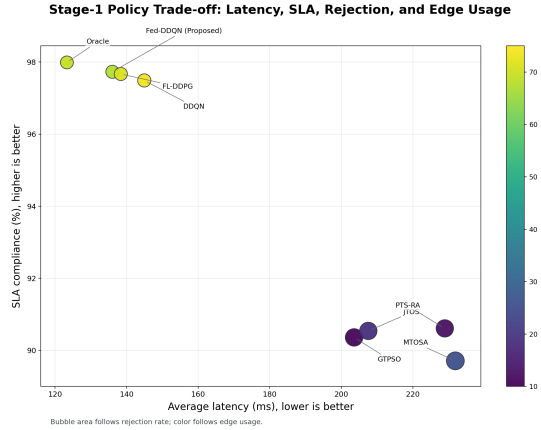


Figure 10: Stage-1 policy tradeoff among latency, SLA satisfaction, rejection, and edge usage.

centralized DDQN, latency is lower by 6.15%, which is consistent with the combined effect of the proposed training, reward, and validation design. Relative to the FL-DDPG-style baseline, latency is lower by 1.71%, showing that the value-based federated design remains competitive against the implemented federated actor-critic-style comparator.

The latency-reference Oracle reaches 123.310 ms. It selects the lower finite generated latency among the edge and cloud alternatives and is not a deployable policy. The gap to this latency reference quantifies remaining latency headroom, while the proposed policy moves closer to that reference than the other implemented main comparison methods and maintains low rejection.

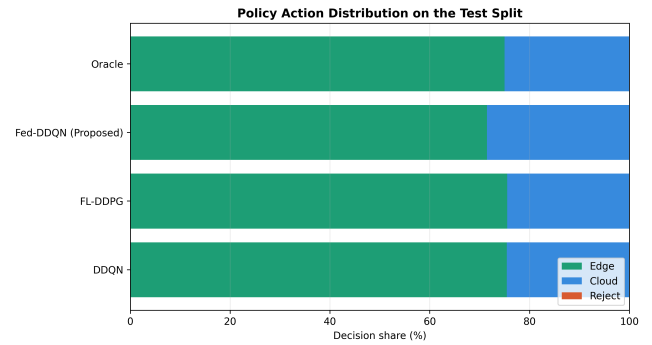


Figure 11: Policy action distribution on the test split for the main comparison methods and the latency-reference Oracle.

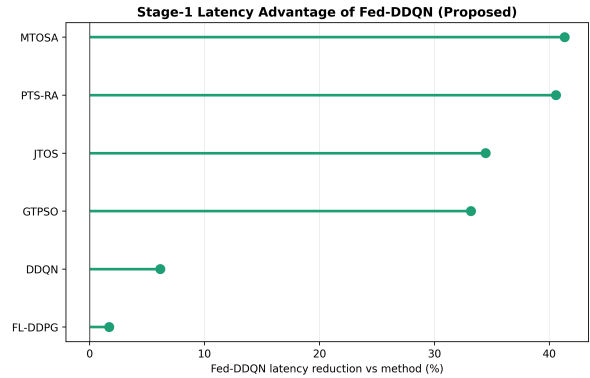


Figure 12: Average latency comparison between trainable policies and the latency-reference Oracle.

Figure 12 gives the same result visually. The heuristic and metaheuristic baselines have much higher latency on this dataset because they do not adapt as directly to the generated channel, queue, and fault states. DDQN improves on those rules but remains more edge-heavy than the proposed policy. The proposed method is not simply choosing cloud or edge more often; it uses a more balanced edge share than DDQN and FL-DDPG while preserving the lowest latency among the main comparison methods in Table 12. Figs. 10 and 11 show the corresponding tradeoff and routing mix on the held-out test interval.

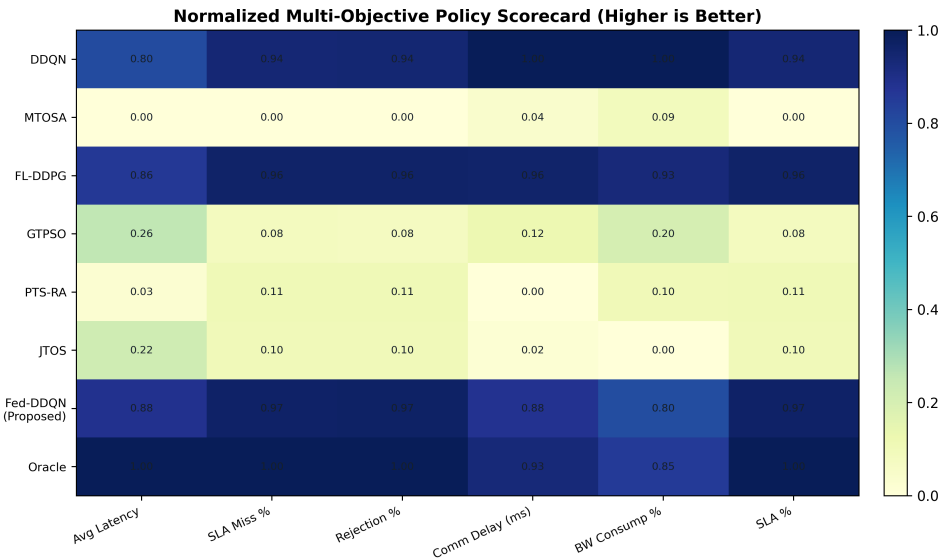


Figure 13: Normalized multi-objective policy scorecard. Higher values indicate stronger normalized performance for the corresponding evaluation criterion.

8.2 QoS and Resource Tradeoff

The SLA and rejection columns are read alongside latency. A low-latency method would not be useful if it obtained that result by rejecting more tasks or overusing a single destination. The proposed method reports 97.73% SLA satisfaction and 3.68% rejection, close to the reported Oracle values of 97.99% SLA satisfaction and 3.43% rejection. These Oracle QoS values are descriptive outputs of the latency-reference rule, not guaranteed QoS bounds. The scorecard in Fig. 13 summarizes this tradeoff.

The edge usage of Fed-DDQN is 67.43%, while Oracle uses 69.27%. This alignment suggests that the reward and validation design reduce the edge overuse that can arise in deadline-heavy workloads. The policy still uses edge for many tight-deadline tasks, but it moves cloud-favorable tasks away from overloaded local nodes.

The action-distribution view in Fig. 11 is included with the QoS discussion because edge share is a resource-stability metric. The tradeoff view in Fig. 10 shows the same balance against latency and SLA behavior. A policy that lowers latency by routing nearly everything to edge would be brittle under bursts; the proposed policy instead stays close to Oracle while keeping rejection low.

8.3 Scenario Behavior and Oracle Agreement

Scenario results are reported before the corresponding figures so the reader can follow the argument without losing the thread. Table 13 uses the same valid held-out policy-evaluation subset as Table 12, so the scenario counts are not full-benchmark descriptive counts. Together with Fig. 14, the table indicates that the learned policy does not behave as a one-rule scheduler. Emergency and real-time slices are routed heavily to edge because local execution often dominates when deadlines are tight. AI/video-heavy and firmware slices shift more work toward cloud because they are larger and are often better served by cloud capacity.

High edge-queue cases show the cost of local contention, with higher latency and SLA miss despite the same learned policy.

Table 13: Fed-DDQN scenario behavior on the valid held-out policy-evaluation subset. Scenario subsets are test/evaluation slices and may overlap.

Scenario	<i>N</i>	Avg. lat.	SLA miss	Edge use
All valid rows	9733	136.855 ± 0.599	2.29 ± 0.05	67.56 ± 0.37
Normal traffic	4061	70.914 ± 0.204	0.26 ± 0.01	67.56 ± 0.33
Congestion	5672	184.066 ± 0.923	3.75 ± 0.09	67.56 ± 0.86
Jitter storm	447	199.934 ± 0.952	2.98 ± 0.10	87.47 ± 0.73
High edge queue	845	245.461 ± 2.105	11.05 ± 0.31	58.50 ± 2.54
Emergency	492	34.049 ± 1.146	22.76 ± 0.17	99.19 ± 0.60
Real-time	4130	43.251 ± 0.319	5.40 ± 0.12	98.41 ± 0.38
Low battery	958	144.930 ± 1.125	3.44 ± 0.09	66.81 ± 1.20
AI/video-heavy	1640	282.075 ± 2.893	0.00 ± 0.00	21.44 ± 0.42
Firmware	251	252.343 ± 2.179	0.00 ± 0.00	33.73 ± 2.73

Note: Values are mean ± standard deviation across the three Fed-DDQN seeds exported for the manuscript. The raw temporal test split contains 11,957 rows; the policy-evaluation subset used for Table 12 contains 9,733 rows after valid generated-latency filtering. Scenario slices are drawn from that valid held-out subset.

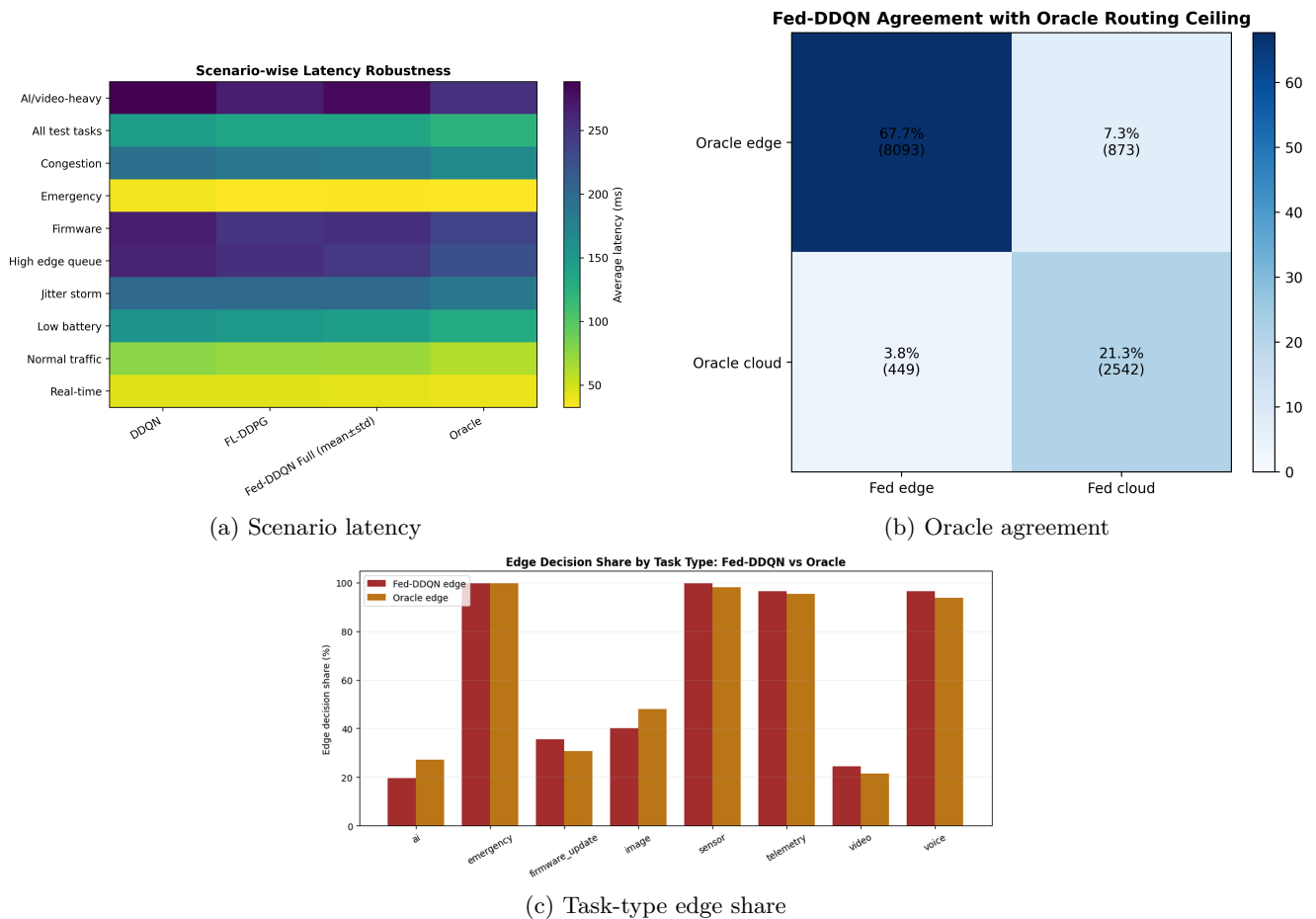


Figure 14: Scenario diagnostics for the proposed policy. Panel (a) compares latency across workload slices, panel (b) shows agreement with the latency-reference Oracle, and panel (c) compares task-type edge decisions with Oracle.

The Oracle-agreement panel in Fig. 14 is a diagnostic rather than a primary score. It identifies where the learned policy still disagrees with the generated latency reference. Disagreement on ambiguous tasks is not automatically wrong, because many cases have similar edge and cloud latency. Disagreement on heavy tasks or emergency tasks matters more because the cost of the wrong destination is larger.

8.4 Multi-Seed Ablation

The multi-seed result reports 136.855 ± 0.599 ms for the proposed configuration across seeds 42, 77, and 123. The low standard deviation indicates stable behavior across the evaluated seeds. The ablations are close in latency, which is expected because the design objective is not latency alone. The action-aware pressure variant has a slightly lower mean latency in the ablation table, while other variants trade latency, rejection, SLA, and edge usage differently. The proposed configuration is retained because it is selected by the balanced validation objective and gives a more controlled edge-usage pattern.

Table 14: Three-seed Fed-DDQN and ablation summary.

Variant	Avg. latency (ms)	SLA (%)	Rejection (%)	Edge usage (%)
Fed-DDQN Proposed	136.855 ± 0.599	97.71 ± 0.05	3.70 ± 0.06	67.56 ± 0.37
Fed-DDQN + Prioritized Replay	137.221 ± 1.089	97.64 ± 0.06	3.77 ± 0.06	66.04 ± 0.33
Fed-DDQN + Action-Aware Pressure	136.393 ± 0.896	97.68 ± 0.07	3.73 ± 0.07	67.50 ± 1.53
Fed-DDQN without FedProx	136.495 ± 0.272	97.65 ± 0.10	3.77 ± 0.10	67.07 ± 1.11
Fed-DDQN with Hard Target Update	137.266 ± 0.501	97.71 ± 0.05	3.70 ± 0.04	67.98 ± 1.08
Fed-DDQN without Heavy-Task Penalty	136.845 ± 0.498	97.68 ± 0.06	3.73 ± 0.05	66.99 ± 2.34

Values are three-seed means with standard deviations from the completed Fed-DDQN ablation export.

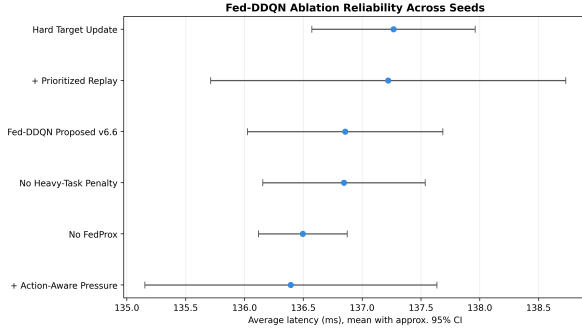


Figure 15: Three-seed latency intervals for the proposed configuration and ablations.

The ablation results support the proposed configuration as a balanced design. Rather than optimizing a single scalar in isolation, the final model balances latency, rejection, SLA satisfaction, and destination usage.

8.5 Resource Allocation

The allocator results explain what happens after the Stage-1 policy selects edge execution. The offloading policy decides where the task goes. The allocator decides how much edge resource to assign after edge is selected. In Table 15, the raw residual allocator reports 0.0504 proxy-target MAE, capacity normalization removes the capacity-violation problem, and the residual plus risk-projection variant reaches 0.0204 proxy-target MAE with zero capacity violation. In the broader comparison, the hand-designed rejection-aware demand rule gives the lowest MAE, while the residual plus risk-projection variant gives the best proxy-target efficiency score, 89.295, the lowest under-allocation, and zero capacity violation.

The raw residual allocator has a notable weakness: it can reduce local error but violate capacity. Capacity normalization fixes that. Risk projection further reduces under-allocation for tasks with higher deadline pressure or QoS/admission risk. The resulting resource manager is more defensible than a pure neural output because the final allocation remains constrained by node capacity.

Stage-2 allocation readout.

$$\begin{aligned}
 \tilde{\mathbf{y}}_i &= \mathbf{b}_i + f_\phi(\mathbf{z}_i), \\
 \mathbf{y}_i^{\text{res}} &= \Pi_{\text{cap}}(\text{clip}_{[0,1]}(\tilde{\mathbf{y}}_i)), \\
 \mathbf{y}_i^{\text{dem}} &= \Pi_{\text{cap}}(\omega_i \mathbf{d}_i^{\text{raw}}), \\
 \mathbf{y}_i^{\text{risk}} &= \Pi_{\text{cap}}(0.10 \mathbf{y}_i^{\text{res}} + 0.90 \mathbf{y}_i^{\text{dem}}), \\
 \hat{\mathbf{y}}_i &= \Pi_{\text{cap}}(\mathbf{y}_i^{\text{risk}}).
 \end{aligned}$$

Here $\mathbf{b}_i$ is the demand-proportional base, $\tilde{\mathbf{y}}_i$ is the residual-corrected allocation, $\mathbf{y}_i^{\text{res}}$ is the capacity-normalized residual allocation, $\mathbf{y}_i^{\text{dem}}$ is the rejection-aware demand branch, $\mathbf{y}_i^{\text{risk}}$ is the blended risk-projected allocation, and $\hat{\mathbf{y}}_i$ is the final capacity-normalized assignment.

Figure 16: Capacity-aware interpretation of the rejection-aware residual allocator using the same Stage-2 target and prediction notation as the allocator equations.

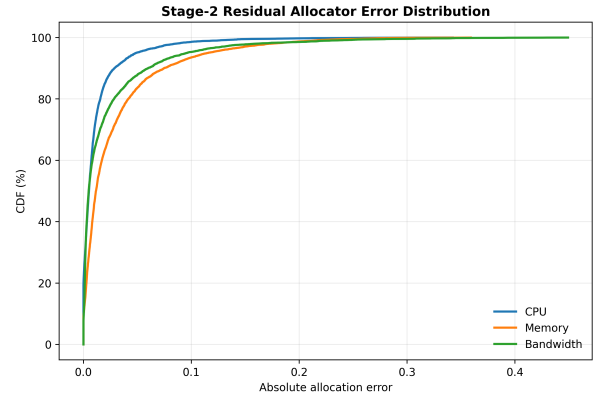


Figure 17: CDF of proxy-target resource-allocation error. Capacity normalization and risk projection reduce the high-error tail relative to raw residual output.

Figure 17 should be read together with Fig. 16. The CDF shows that most resource errors are small after residual correction, while the projection equation explains how the final assignment is kept inside the feasible capacity region. This is the main reason the residual allocator is reported with both accuracy and feasibility metrics: low prediction error is useful only when the assigned CPU, memory, and bandwidth remain admissible for the selected edge node.

The practical reading is straightforward. Demand-proportional allocation gives a stable starting point, the residual model adjusts for deadline and priority conditions, and the risk term protects tasks that are likely to miss QoS after edge selection. Capacity

Table 15: Stage-2 allocator proxy-target error and feasibility on $\mathcal{T}_e^{\text{valid}}$, the Fed-DDQN edge-selected allocator-valid test subset.

Method	Overall MAE	Underalloc. (%)	Waste (%)	Cap. viol. (%)
Equal Allocation	0.3030	16.59	57.01	0.00
Demand-Proportional	0.0656	23.56	16.93	0.00
Priority-Weighted	0.2808	10.89	47.09	0.00
Deadline-Aware	0.3268	23.06	52.60	0.00
Rejection-Aware Demand	0.0196	6.98	5.46	0.00
Residual Allocator Raw	0.0504	11.39	19.24	35.16
Residual + Capacity Norm	0.0366	15.23	12.12	0.00
Residual + Risk Projection	0.0204	6.65	5.75	0.00

Table 16: Stage-2 allocator proxy-target and QoS-oriented comparison on the allocator-valid edge-selected subset.

Method	Est. SLA risk (%)	Efficiency	Interpretation
Equal Allocation	4.57	58.319	Balanced but wastes resource because task demand is ignored.
Demand-Proportional	5.96	66.692	Low error relative to simple baselines, but under-allocation remains high.
Priority-Weighted	3.31	68.623	Reduces risk for urgent tasks but wastes more resource.
Deadline-Aware	3.12	54.338	Protects deadlines but over-allocates and loses efficiency.
Rejection-Aware Demand	3.55	89.062	Best MAE and waste, using the hand-designed risk-aware demand rule.
Residual Allocator Raw	4.08	57.773	Neural residual learns useful corrections but violates capacity before projection.
Residual + Capacity Norm	3.61	78.120	Removes capacity violations and improves feasibility.
Residual + Risk Projection	3.50	89.295	Best efficiency, lowest under-allocation, and zero capacity violation.

projection then prevents the learned correction from turning into an infeasible assignment during high-load intervals.

8.6 Discussion

The results support the proposed two-stage structure. Fed-DDQN is the main contribution because it improves the edge/cloud scheduling decision on the benchmark dataset. The allocator is a secondary contribution because it improves proxy-target allocation fit for the allocator-valid edge-selected subset after the scheduler has already acted. Keeping these roles separate makes the result easier to interpret.

The result also shows the value of the benchmark dataset. The scenario-wise and workload-timeline figures indicate that the policy is exposed to bursts, congestion, outage, and resource variation. This stress structure makes the benchmark suitable for comparing offloading policies under repeatable but demanding conditions.

Finally, the latency-reference Oracle gap identifies directions for future optimization. The learned policy remains about 12.7 ms above that generated latency reference, leaving room for recurrent policies, graph-based edge-state encoders, and online state mutation where policy actions affect subsequent queues directly.

8.7 Interpretation and Practical Implications

The proposed federated DDQN configuration achieves the best average latency among the main comparison methods in Table 12 while keeping rejection and SLA behavior close to the reported Oracle values. This combination matters for edge-cloud IoT scheduling

because a policy must reduce delay without shifting difficult tasks into rejection or overloading one execution tier.

The main qualitative result is the reduction of edge bias. Deadline-heavy workloads can make a policy route too many tasks to edge, which helps emergency tasks but hurts AI, video, image, and firmware-like cases. The reward and validation design bring edge usage closer to the Oracle level. That is why the paper reports edge usage, scenario behavior, and Oracle agreement rather than only latency. The method is not simply using one destination more often.

The allocator result follows the same staged logic. Residual learning improves proxy-target allocation fit on the allocator-valid edge-selected subset, and the best allocator result comes from combining learned correction with risk projection and normalization. For resource management, the constrained final prediction is preferable because it keeps the allocation feasible while improving proxy-target error and QoS-oriented efficiency.

9 SCOPE AND VALIDITY CONSIDERATIONS

The study uses the benchmark dataset as a realistic synthetic benchmark with explicit burst, congestion, outage, degradation, and maintenance windows. Race-condition pressure is represented through precomputed contention and queue carryover instead of live mutable queues. This choice keeps the comparison repeatable and the evaluation auditable.

The compared methods are implemented inside the same evaluation pipeline for comparison fairness. They

share the same features, temporal split, and metrics, which keeps the evaluation aligned across policies. The FL-DDPG-style method serves as the federated actor-critic comparator within this common pipeline.

The multi-seed analysis uses three seeds. It provides a repeatability check for the proposed configuration and ablations. Future extensions can add more seeds, confidence intervals for all baselines, independent workload-generator samples, and a no-bootstrapping ablation with $\gamma = 0$ to isolate the value of temporal bootstrapping. No unrun baseline-variance or $\gamma = 0$ results are included in the present tables.

Federated training is privacy-aligned because raw zone data are not pooled in the algorithmic design. Secure aggregation, differential privacy, and attack evaluation against model updates remain outside the present implementation and are natural extensions for a security-focused study. Communication efficiency is therefore treated as configuration-dependent; the present manuscript reports the transfer expression but does not claim that model-update traffic is always smaller than centralizing all raw features.

Oracle and supervised label generation depend on the same latency functions and state tables used by evaluation. This keeps the benchmark internally consistent and makes Oracle a useful generated latency reference. Measured-trace studies can extend the same evaluation protocol with deployment-specific overheads and online queue mutation.

The pipeline evaluates offloading decisions in an offline replay style. The dataset contains precomputed contention, while a future live simulator could allow each policy action to mutate subsequent queues, bandwidth pressure, and admission state. This would extend the benchmark into an online closed-loop evaluation setting and would test whether the same reward and validation design remains stable when actions change future states rather than only current-task rewards.

10 CONCLUSION

The work presented a two-stage edge-cloud IoT scheduling method. The primary contribution is a zone-aware Fed-DDQN policy for binary edge/cloud task offloading. The secondary contribution is a rejection-aware residual allocator for CPU, memory, and bandwidth assignment after edge execution is selected.

On the benchmark dataset, the proposed policy achieved 136.009 ms average test latency, 97.73% SLA satisfaction, and 3.68% rejection. It improved latency by 6.15% relative to centralized DDQN and by 1.71% relative to the FL-DDPG-style baseline, while the latency-reference Oracle remained lower at 123.310 ms. On the allocator-valid edge-selected subset, the hand-designed rejection-aware demand rule gave the lowest proxy-target MAE, while residual learning with risk projection and capacity normalization gave the best efficiency score, 89.295, the lowest under-allocation, and zero capacity violation.

The practical implication is that offloading should be treated as a channel-aware and resource-aware control problem, not a single latency-minimization decision. In

burst windows, an edge node can be attractive for one urgent task and still be a poor destination for a group of heavy tasks arriving at the same time. The proposed policy uses state variables that reflect local edge pressure, cloud delay, packet loss, bandwidth, task priority, and deadline sensitivity. As a result, the learned decision can change when the same task profile appears under different network or queue states.

Table 17 summarizes the deployment reading of the final scheduler.

Table 17: Operational reading of the two-stage scheduler.

Condition	Stage-1 offloading response	Stage-2 allocation response
Queue burst or local contention	Reduce unnecessary edge selection and favor cloud when local delay risk dominates.	Preserve bounded edge resources for accepted urgent tasks.
Channel degradation or packet loss	Use network-state features to avoid cloud paths with poor transmission quality.	Avoid over-allocating bandwidth when the selected path is already channel-limited.
Tight deadline and high priority	Prefer the destination with lower predicted service delay and lower QoS/admission risk.	Increase correction weight for deadline-sensitive tasks within node capacity.
Heavy video, AI, or firmware load	Avoid treating all large tasks as edge-favorable merely because edge execution can be fast.	Normalize CPU, memory, and bandwidth so heavy tasks do not starve nearby tasks.

The table separates what the proposed method learns from how it can be deployed. Stage 1 selects an execution tier and is penalized when the selected path is infeasible under changing channel and queue conditions. Stage 2 turns an accepted edge decision into resource shares that are bounded, risk-aware, and comparable across task classes.

The federated structure also matters in deployment. Instead of assuming that all edge-zone traces can be pooled centrally, the training loop allows zone-level learners to update locally and share model parameters through aggregation. This matches IoT deployments where traffic patterns vary across urban, industrial, rural, and mobile edge zones. The global model captures common offloading behavior while still benefiting from heterogeneous local experience. FedProx-style regularization, validation checkpointing, and the balanced reward design stabilize learning when local workload distributions differ.

The Stage-2 allocator matters after the offloading decision is made. Edge selection alone does not guarantee that a task receives enough CPU, memory, and bandwidth to match the generated proxy target. The residual allocator starts from demand-proportional assignment, learns corrections for deadline, priority, and QoS/admission-risk conditions, and then applies capacity normalization so the final allocation remains feasible. This separates the routing question from the resource-admission question while keeping both stages tied to the same QoS objective.

In an operational edge platform, the same structure can be used as a layered scheduler. The Stage-1 policy selects the execution tier using current task, channel, and node-state information. The Stage-2 allocator then converts an edge decision into bounded resource shares that respect node capacity. This modularity makes the design easier to monitor because routing metrics and resource-admission metrics can be inspected separately while still being optimized toward a common QoS objective.

Taken together, the results show that federated DDQN-based offloading and rejection-aware adaptive allocation form a coherent scheduling pipeline for edge-cloud IoT systems. The method reduces latency against the implemented main comparison methods, preserves high SLA satisfaction, controls rejection, and provides interpretable diagnostics through scenario, ablation, and allocator analyses. Future work can extend this pipeline with measured deployment traces, secure aggregation, additional workload generators, and a closed-loop simulator in which policy actions update subsequent queue and resource states.

A ADDITIONAL DIAGNOSTICS

This appendix collects diagnostics that support the main results in Table 12. Fig. 18 and Fig. 19 connect workload stress to the main policy metrics. Figs. 20–24 explain the Stage-2 allocator behavior behind Tables 15 and 16. Table 18 lists the task-type SLA thresholds used for SLA and deadline-related QoS evaluation.

A.1 Policy Stress and Metric Reading

The edge queue response in Fig. 18 explains why the offloading state includes queue-aware features. Bursty arrivals change the local edge condition for multiple tasks at the same or nearby timesteps, so the policy cannot rely only on task size or deadline.

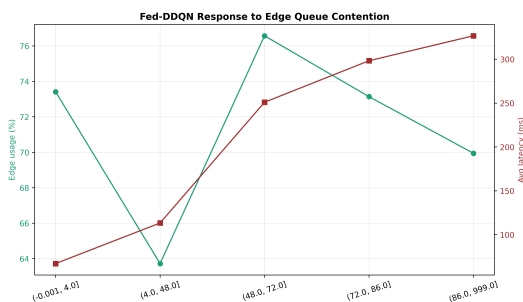


Figure 18: Edge queue pressure response under workload bursts.

The pattern to notice in Fig. 18 is the persistence of queue pressure after a burst begins. It supports the use of temporal train-validation-test splitting and queue-related state variables: a policy trained only on independent task samples would not observe the same delayed effect of contention on nearby tasks. It also explains why edge selection must be moderated during burst windows even when individual tasks look edge favorable in isolation.

Core Test Metrics Snapshot				
	Latency ms	SLA %	Rejection %	Edge use %
DDQN	144.9	97.49	3.90	73.71
FL-DDPG	138.4	97.67	3.74	71.23
Fed-DDQN	136.0	97.73	3.68	67.43
Oracle	123.3	97.99	3.43	69.27

Green means better for the metric: lower latency/rejection, higher SLA, edge use closer to Oracle.

Figure 19: Core test-metric snapshot for the closest trainable comparators and the latency-reference Oracle. Green cells indicate the preferred direction for each metric: lower latency and rejection, higher SLA, and edge usage closer to Oracle.

The metric snapshot in Fig. 19 gives a direct reader-facing summary of the main policy tradeoff. It keeps the proposed model beside DDQN, FL-DDPG, and Oracle, so the latency gain can be read together with SLA, rejection, and destination balance. Stage 1 is judged by these policy metrics, while Stage 2 is judged separately by allocation error, under-allocation, waste, capacity violation, and risk-aware efficiency. The two views should not be collapsed into one number.

A.2 Stage-2 Allocator Diagnostics

The allocator diagnostics separate error, feasibility, and QoS behavior. A method with low mean error can still be poor if it violates capacity or under-allocates deadline-sensitive tasks. Fig. 20 summarizes the tradeoff across all allocator variants. Fig. 21 shows the under-allocation and excess profile after capacity normalization. Fig. 22 compares target and assigned resource distributions, while Fig. 23 shows which task types enter Stage 2 after the proposed offloading policy selects edge execution.

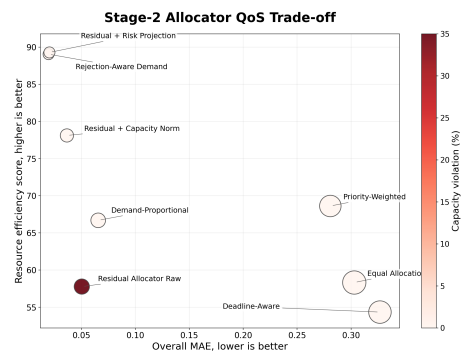


Figure 20: Allocator tradeoff between resource error, under-allocation, waste, SLA risk, and efficiency.

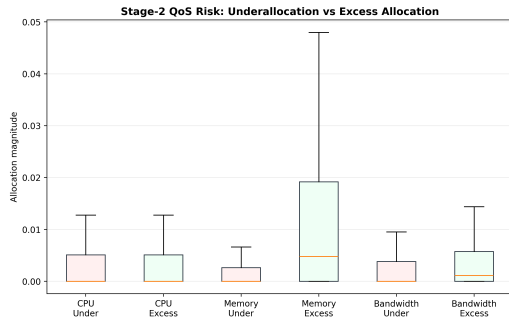


Figure 21: Allocator under-allocation and excess profile. Capacity normalization prevents infeasible raw residual outputs from becoming final assignments.

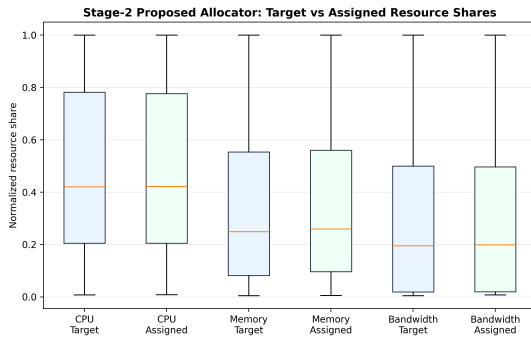


Figure 22: Distribution of allocator targets and assigned resources for Stage-2 evaluation.

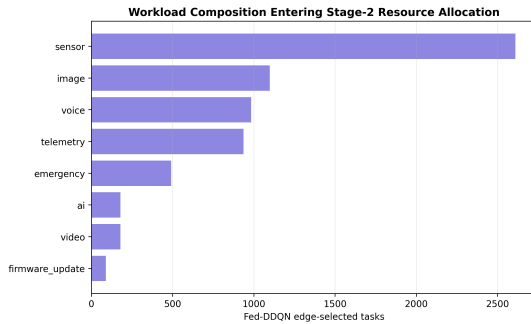


Figure 23: Task mix among the edge-selected tasks passed to the Stage-2 allocator.

Table 18: Task-type SLA thresholds used for QoS evaluation.

Task type	SLA threshold (ms)
Emergency	50
Voice	150
Sensor	500
Telemetry	800
Image	1500
Video	3000
AI	5000
Firmware update	60000

The SLA thresholds in Table 18 explain why the allocator evaluation reports both average resource error and QoS-oriented measures. Emergency and voice tasks are sensitive to short delays, whereas firmware, video, and AI workloads tolerate longer execution windows but can consume more capacity. This task-dependent tolerance is the reason the Stage-2 results are reported

with under-allocation, waste, SLA-risk, and efficiency metrics instead of a single aggregate error value.

The ablation plot in Fig. 24 complements the allocator tables by showing that the final residual-plus-risk-projection variant is not only a lower-error model. It combines learned correction with capacity-safe projection, keeping the final allocator assignment feasible while reducing high-risk under-allocation cases.

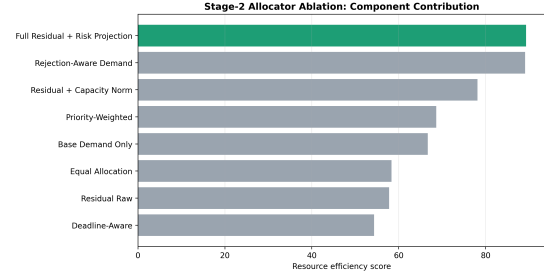


Figure 24: Allocator ablation efficiency comparison.

REFERENCES

- [1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017. [Online]. Available: <https://doi.org/10.1109/COMST.2017.2745201>
- [2] C. Feng, P. Han, X. Zhang, B. Yang, Y. Liu, and L. Guo, "Computation offloading in mobile edge computing networks: A survey," *Journal of Network and Computer Applications*, vol. 202, p. 103366, May 2022. [Online]. Available: <https://doi.org/10.1016/j.jnca.2022.103366>
- [3] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016. [Online]. Available: <https://doi.org/10.1109/TNET.2015.2487344>
- [4] C. You, K. Huang, H. Chae, and B.-H. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Transactions on Wireless Communications*, vol. 16, no. 3, pp. 1397–1411, Mar. 2017. [Online]. Available: <https://doi.org/10.1109/TWC.2016.2633522>
- [5] S. Sardellitti, G. Scutari, and S. Barbarossa, "Joint optimization of radio and computational resources for multicell mobile-edge computing," *IEEE Transactions on Signal and Information Processing over Networks*, vol. 1, no. 2, pp. 89–103, Jun. 2015. [Online]. Available: <https://doi.org/10.1109/TSIPN.2015.2448520>
- [6] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Transactions on Mobile Computing*, vol. 21, no. 6, pp. 1985–1997, Jun. 2022. [Online]. Available: <https://doi.org/10.1109/TMC.2020.3036871>
- [7] I. Ullah, H.-K. Lim, Y.-J. Seok, and Y.-H. Han, "Optimizing task offloading and resource allocation in edge-cloud networks: A DRL approach," *Journal of Cloud Computing*, vol. 12, p. 112, 2023. [Online]. Available: <https://doi.org/10.1186/s13677-023-00461-3>
- [8] H. Wu, J. Geng, X. Bai, and S. Jin, "Deep reinforcement learning-based online task offloading in mobile edge computing networks," *Information Sciences*, vol. 656, p. 119849, Jan. 2024. [Online]. Available: <https://doi.org/10.1016/j.ins.2023.119849>
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: <https://doi.org/10.1038/nature14236>
- [10] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with Double Q-Learning," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 30, no. 1, 2016, pp. 2094–2100. [Online]. Available: <https://doi.org/10.1609/aaai.v30i1.10295>
- [11] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 48, 2016, pp. 1995–2003. [Online]. Available: <https://proceedings.mlr.press/v48/wangf16.html>
- [12] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1509.02971>
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. Proc. Mach. Learn. Res., vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [14] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, May 2020. [Online]. Available: <https://doi.org/10.1109/MSP.2020.2975749>
- [15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html
- [16] J. Li, Z. Yang, X. Wang, Y. Xia, and S. Ni, "Task offloading mechanism based on federated reinforcement learning in mobile edge computing," *Digital Communications and Networks*, vol. 9, no. 2, pp. 312–321, Apr. 2023. [Online]. Available: <https://doi.org/10.1016/j.dcan.2022.04.006>
- [17] X. Li, J. Zhang, and C. Pan, "Federated deep reinforcement learning for energy-efficient edge computing offloading and resource allocation in industrial Internet," *Applied Sciences*, vol. 13, no. 11, p. 6708, 2023. [Online]. Available: <https://doi.org/10.3390/app13116708>
- [18] Z. Tong, J. Wang, J. Mei, K. Li, W. Li, and K. Li, "Multi-type task offloading for wireless Internet of

- Things by federated deep reinforcement learning,” *Future Generation Computer Systems*, vol. 145, pp. 541–556, Aug. 2023. [Online]. Available: <https://doi.org/10.1016/j.future.2023.04.004>
- [19] X. Zhao, Y. Wu, T. Zhao, F. Wang, and M. Li, “Federated deep reinforcement learning for task offloading and resource allocation in mobile edge computing-assisted vehicular networks,” *Journal of Network and Computer Applications*, vol. 226, p. 103941, May 2024. [Online]. Available: <https://doi.org/10.1016/j.jnca.2024.103941>
- [20] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1511.05952>